香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC6910J
Deep Reinforcement Learning

Lecture 13. Exam Review

Course Instructor: Ling PAN
Department of Electronic and Computer Engineering
Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188, CMU 10-403, UCL COMPM050

Announcement

- In-class exam: 15:00-17:50, Oct 23, Room 6555
  - A one-page (single-sided) cheatsheet is allowed.
  - Calculator is allowed.
  - Please contact the TAs **in advance** is you have a time conflict.

ELEC6910J **Core Checklist for the Exam**

Time & Date: 15:00-17:40, Oct. 23 (in-class exam)
Venue: Room 6555 (our classroom)
Question types: multiple-choice, fill-in-the-blanks, calculations
Note: Closed-book exam; however, *you can bring one double-sided A4 sheet of notes*.

Bandits & Exploration

- ☐ The exploration vs. exploitation trade-off
- ☐ ϵ-greedy exploration strategy
- ☐ Stationary vs. non-stationary bandits

Markov Decision Process (MDP)

- ☐ Definition & understanding
- ☐ The role of the discount factor γ and its impact on optimal policies

Value Functions and Bellman Equations

- ☐ Recursive relationships between current and future values
- ☐ State-value functions and action-value functions
- ☐ Bellman expectation equations and Bellman optimality equations

Dynamic Programming

- ☐ Value Iteration: update equations and finding the optimal policy
- ☐ Policy Evaluation: iterative calculation of state values for a given policy
- ☐ Policy Iteration: policy evaluation and policy improvement steps

Monte Carlo Methods

- ☐ First/Every-visit Monte Carlo prediction and value estimation
- ☐ Characteristics of Monte Carlo methods

Temporal Difference Learning

- ☐ Calculation of the TD target and TD error (including multi-step TD)
- ☐ Monte Carlo vs. TD learning: fundamental differences, bias, and variance
- ☐ Differences between on-policy and off-policy learning approaches
- ☐ SARSA: On-policy control algorithm
- ☐ Q-learning: Off-policy control algorithm

General Machine Learning Concepts

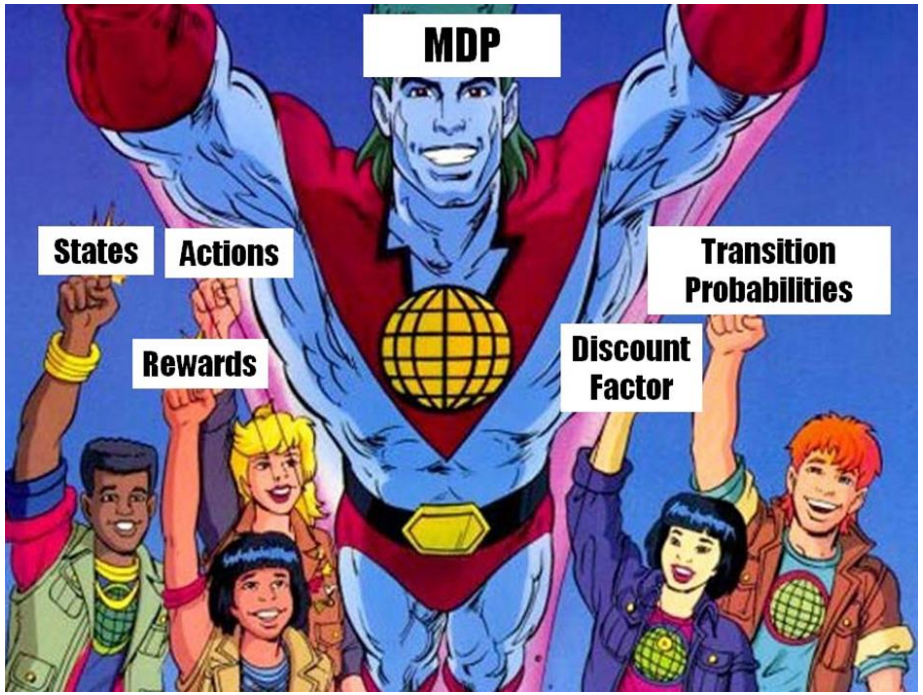
- ☐ Differences between supervised learning and reinforcement learning
- ☐ Differences among DP, MC, and TD

Practical Problem-Solving

- ☐ Step-by-step tabular value updates, deriving optimal policies
- ☐ Matrix and algebraic calculations

Markov Decision Process (MDP)

- Definition: (S, A, T, R, γ)
  - A set of states $s \in S$
  - A set of actions $a \in A$
  - A transition function $T(s, a, s')$
    - Probability that a from s leads to s', i.e., $P(s'|s, a)$
    - Also called the model or the dynamics
  - A reward function $R(s, a, s')$
  - A discount factor γ



- Quantities:
  - Policy: map of states to actions
  - Utility: sum of discounted rewards
  - State values: expected future utility from a state
  - State-action (Q-)values: expected future utility from a state-action pair

Value Functions

- **Recursive** relationships for returns

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$
$$= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots)$$
$$= R_{t+1} + \gamma G_{t+1}$$

Condition on a state and take expectations

$$\mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s]$$

$V^\pi(s)$: value of a state

$\mathbb{E}_\pi[R_{t+1}$: expected immediate reward

$+\gamma V^\pi(S_{t+1})$: +discounted value of next state

$| S_t = s]$: if the agent starts at state s

and uses the policy to choose its actions for all time steps

(law of total probability)

$$= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V^\pi(s')]$$

Value Functions

- Bellman equation for V^{π}:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V^{\pi}(s')]$$

  - Relationship between the value of the current state and the values of its successor states
  - The value function $V^{\pi}(s)$ is the unique solution to its Bellman equation



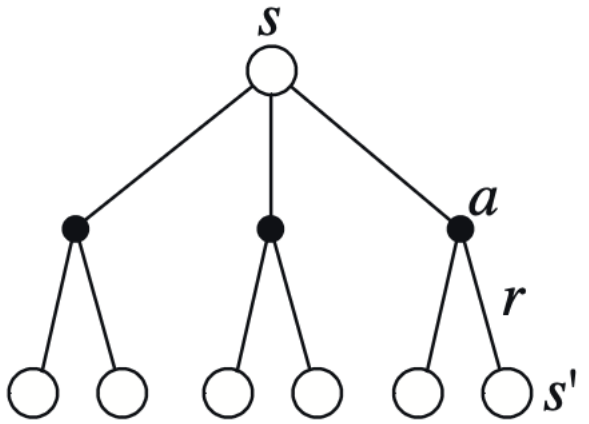
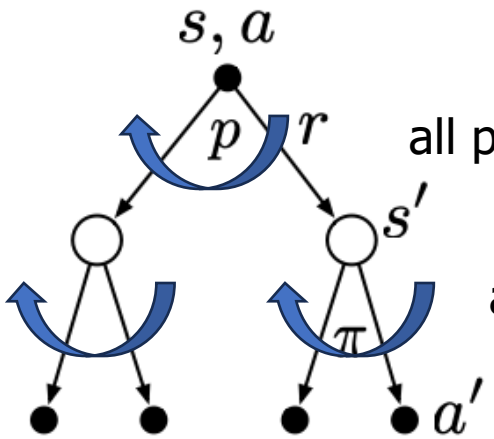


Figure. Backup diagram.

Value Functions

- Bellman equation for Q^π:

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left[r + \gamma \sum_{a'} \pi(a'|s')\, Q^\pi(s',a')\right]$$



all possible next states and rewards listed with their probabilities

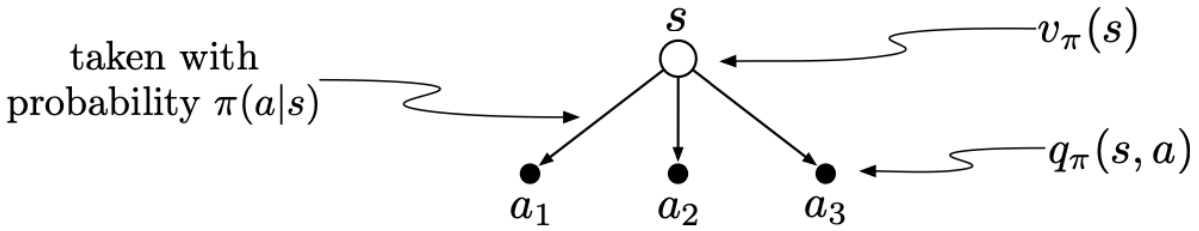
all next actions listed with their probabilities

Figure. Backup diagram.

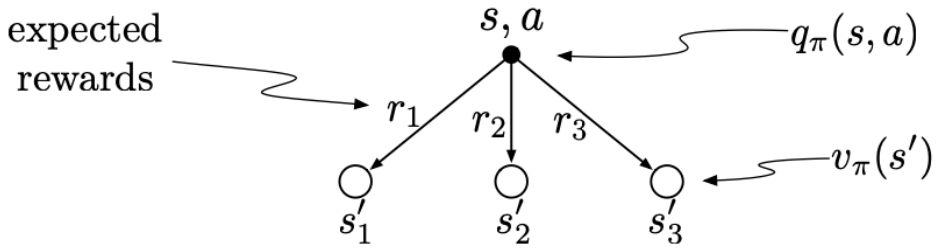
Value Functions

- Relating V^π and Q^π:

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$$



$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a)[r + \gamma V^\pi(s')]$$



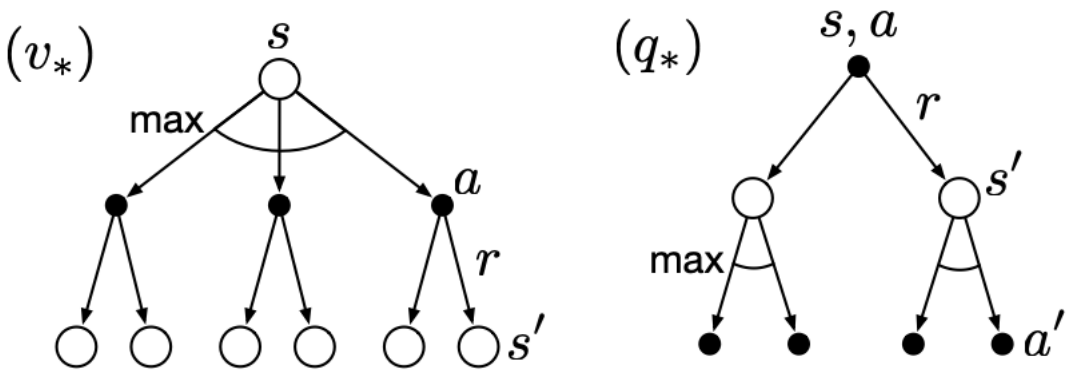
Optimal Value Functions

- Bellman optimality equation for V^*:
  - The value of a state under an optimal policy must equal the expected return for the best action from that state

$$V^*(s) = \max_a \left(\sum_{s',r} p(s', r|s, a)[r + \gamma V^*(s')] \right)$$

- Bellman optimality equation for Q^*:

$$Q^*(s, a) = \sum_{s',r} p(s', r|s, a) \left[r + \gamma \max_{a'} Q^*(s', a') \right]$$



$V^*(s)$ and $Q^*(s, a)$ is the unique solution of this system of nonlinear equations

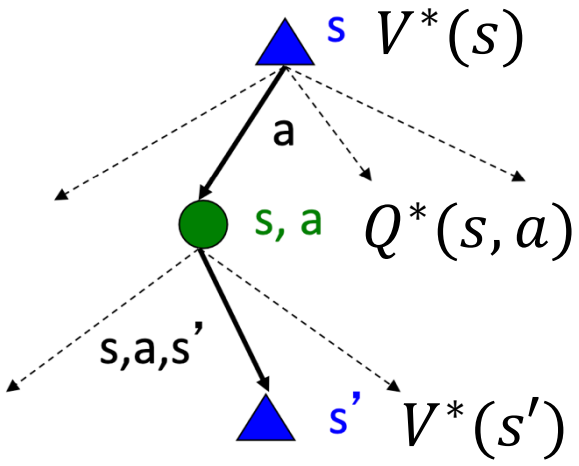
Optimal Value Functions

- Recursive definition of value (Bellman equation)
  - Optimal state value function:

$$V^*(s) = \max_a Q^*(s,a)$$

  - Optimal state-action value function:

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$



- Optimal policy

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$$

Value Iteration

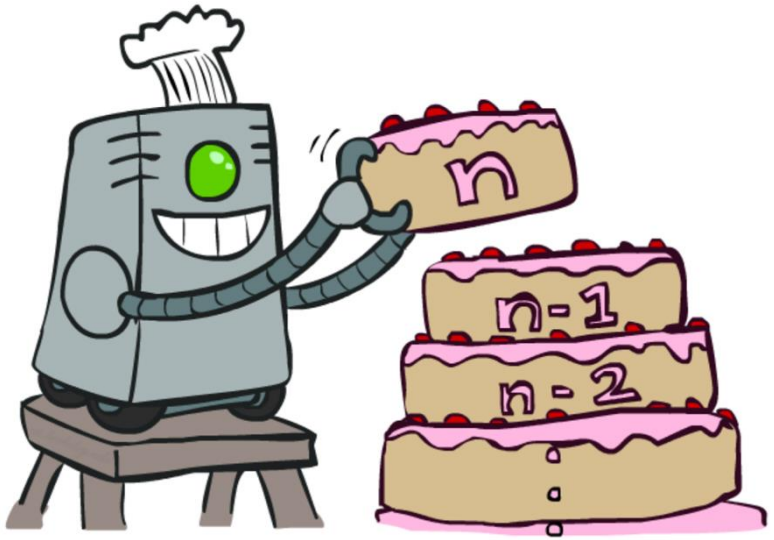
- Bellman equation characterizes the optimal values

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

- Value iteration computes the optimal values

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

  - A fixed point solution method

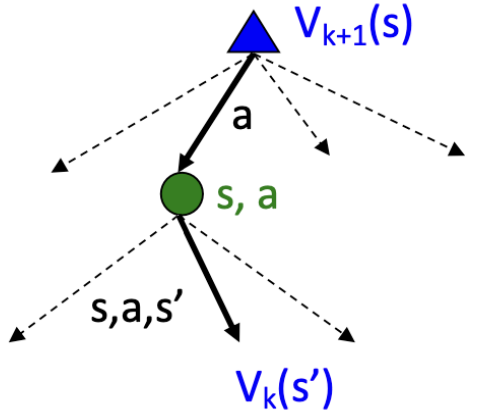


Value Iteration

- Start with $V_0(s) = 0$: zero utility if the game ends now
- Given vector of $V_k(s)$ values, we can compute $V_{k+1}(s)$

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

- Repeat until convergence, which yields V^*

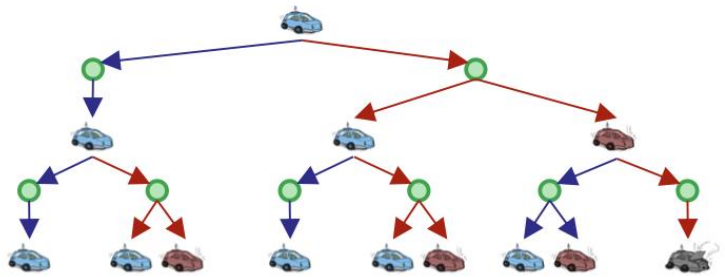


- The complexity of each iteration in terms of state space cardinality $|S|$ and action space cardinality $|A|$
  - $O(|S|^2|A|)$ – as you need to operate on every state

- Theorem: will converge to unique optimal values
  - Basic idea: approximations get refined towards optimal values
  - Policy may converge long before values do

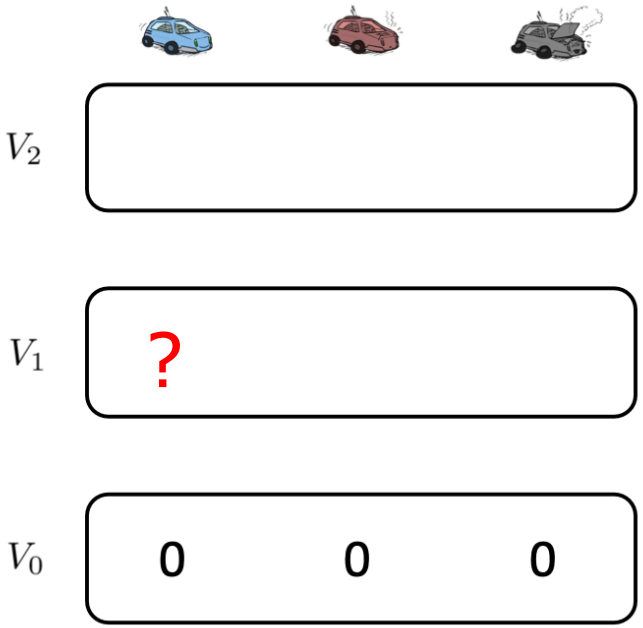
Value Iteration

- Example: Car racing

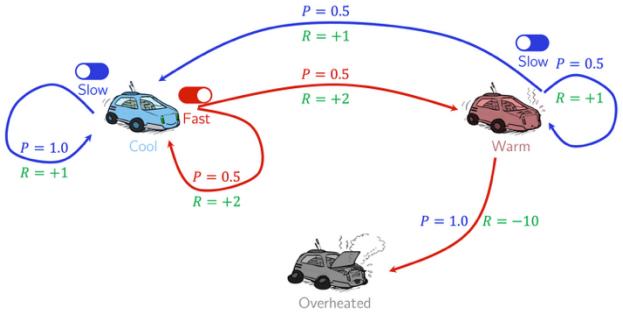


s	a	s'	$P(s'\|s,a)$	$R(s,a,s')$
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')] \quad \text{(assume no discount)}$$



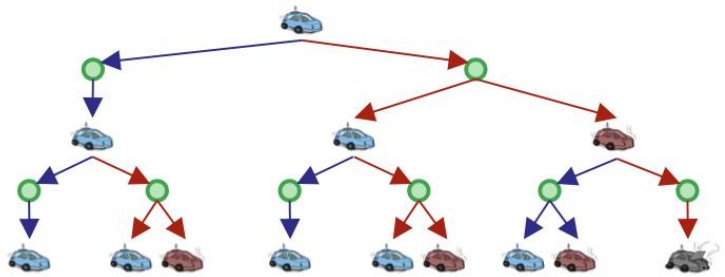
- action=slow:
  $1\times(1+0)=1$
- action=fast:
  $0.5\times(2+0)+0.5\times(2+0)=2$



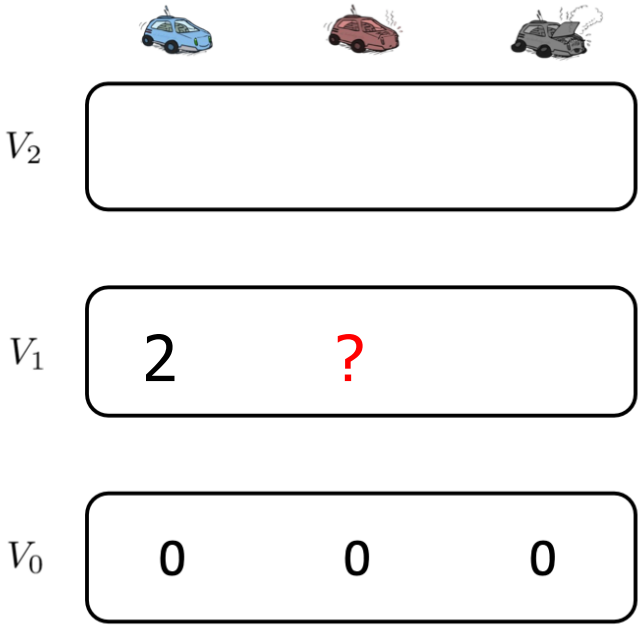
Value Iteration

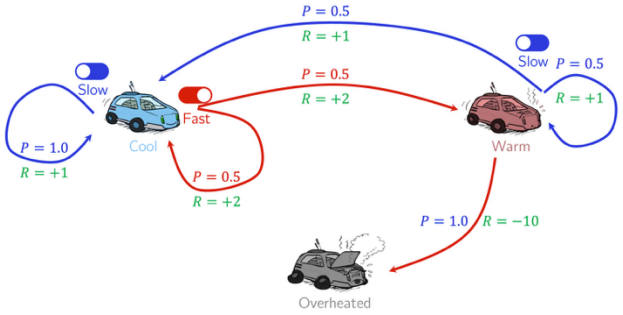
- Example: Car racing

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')] \quad \text{(assume no discount)}$$



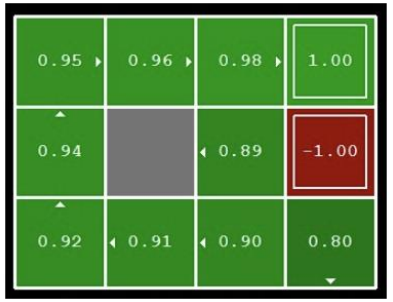
s	a	s'	$P(s'\|s,a)$	$R(s,a,s')$
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0





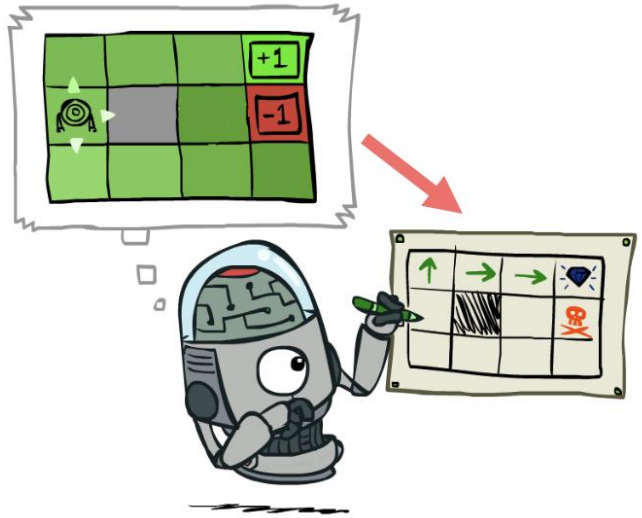
Computing Actions from Values

- Problem: how do we get actions?
  - Imagine we have the optimal values $V^*(s)$
  - How should we act?



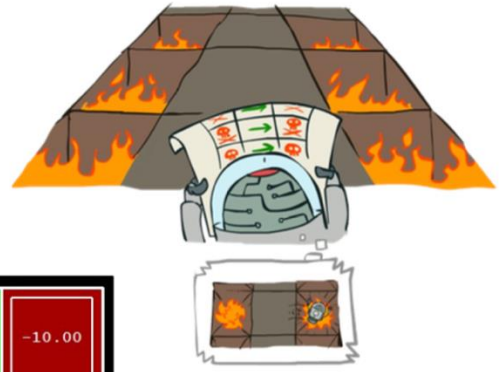
- Policy extraction: gets the policy implied by the values
  - We need to do a one-step lookup

$$\pi^*(s) = \text{argmax}_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$



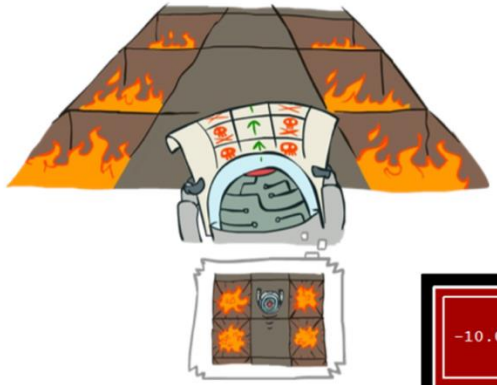
Policy Evaluation

Always Go Right





Always Go Forward





Usage: computing the value function for a fixed policy is to help find better policies.

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- Efficiency: $O(|S|^2)$ per iteration

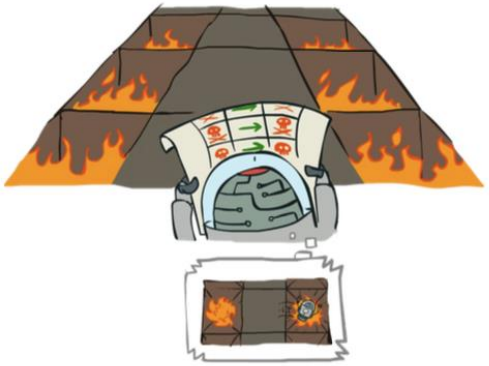
Policy Improvement

- For a given state s,

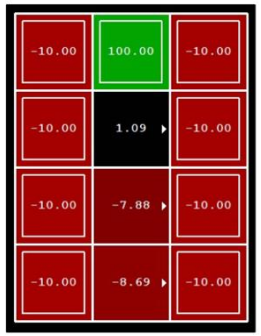
  would it be better to do an action $a \neq \pi(s)$ and thereafter follow π?

  - We know how good it is to follow the current policy π from $s \rightarrow V^{\pi}(s)$
  - Would it be better or worse to change to a new policy?

Policy π: Always Go Right



Value function V^{π}



New policy:

- Go forward
- Follow π (always go right) thereafter





Policy Iteration

- Evaluation: For fixed current policy π, find values with policy evaluation:
  - Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') \left[R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s') \right]$$

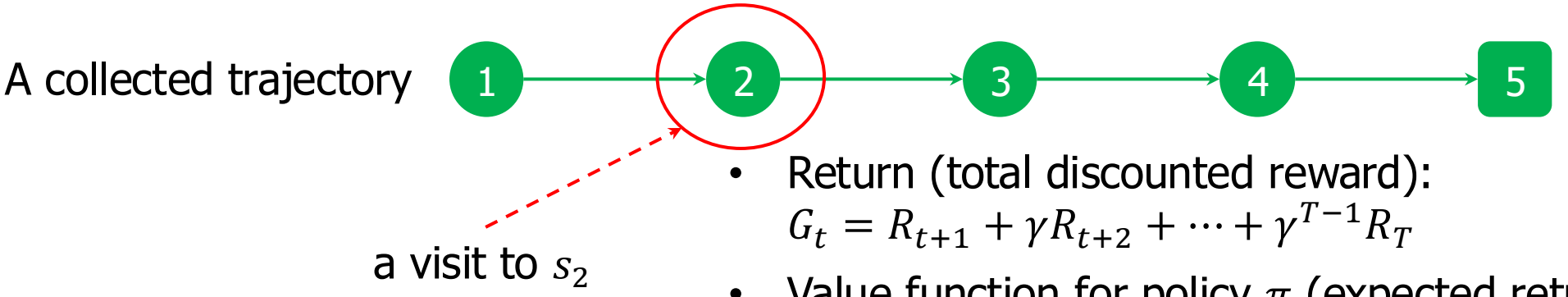
  - End up with value function
- Improvement: For fixed values, get a better policy using policy extraction
  - One-step look-ahead:

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^{\pi_i}(s') \right]$$

- Repeat steps until policy converges

Monte-Carlo Prediction

- Goal: learn $V^{\pi}(s)$ from episodes of experience under policy π
- Method: average returns observed after visits to s



- Return (total discounted reward):
  $G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-1} R_T$
- Value function for policy π (expected return):

  $V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$ starting at state s

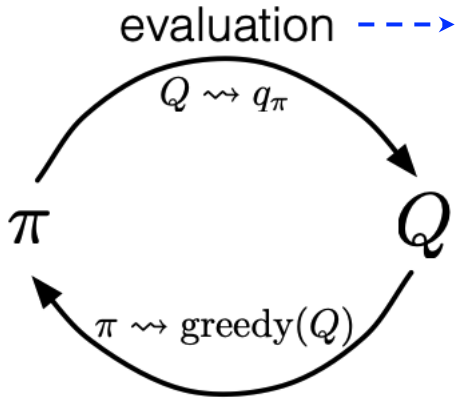
  value of a state — expected discounted return
- Monte-Carlo policy evaluation uses empirical mean return instead of expected return

- First-visit MC: average returns only for first time s is visited in an episode
- Every-Visit MC: average returns for every time s is visited in an episode

Monte-Carlo Control

- Consider a Monte-Carlo version of classical policy iteration

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*$$



MC policy iteration step:
Policy evaluation using MC methods
(followed by policy improvement)

Policy improvement step:
Greedify with respect to Q-function

$$\pi(s) = \text{argmax}_a Q(s,a)$$

- Meaning: for each s, deterministically chooses an action with maximal Q-value
- Effect: construct each π_{k+1} as the greedy policy w.r.t. Q_k^π

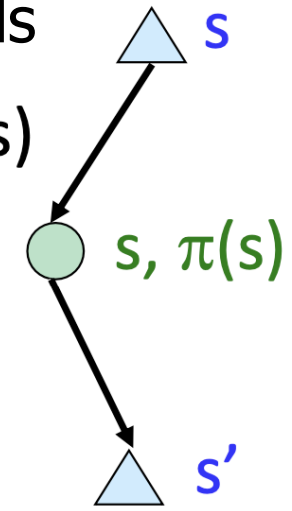
$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg\max_a q_{\pi_k}(s,a)) \\ &= \max_a q_{\pi_k}(s,a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \end{aligned}$$

Temporal Difference Prediction

$$V(S_t) \leftarrow V(S_t) + \alpha[\overbrace{R_{t+1} + \gamma V(S_{t+1}) - V(S_t)}^{\text{TD error } \delta_t}]$$

$R_{t+1} + \gamma V(S_{t+1})$: TD target: an estimate of the return

- ✓ TD methods learn directly from episodes of experience
- ✓ TD is model-free: no knowledge of MDP transitions / rewards
- ✓ TD learns from incomplete episodes, by bootstrapping
- ✓ TD updates a guess towards a guess



Temporal Difference Prediction

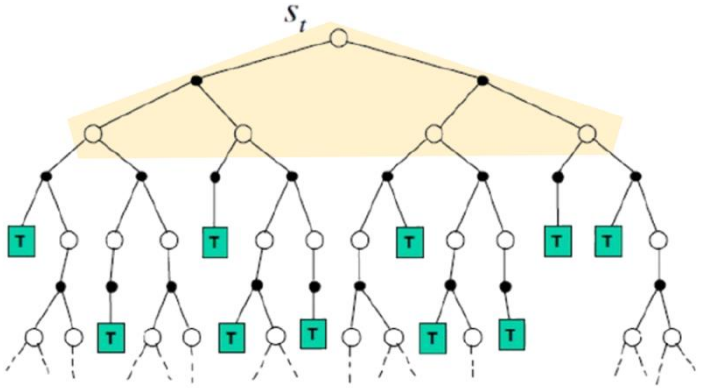
MC:
- The expected value is unknown
- A sample return is used in place of the real expected return

$$
\begin{aligned}
v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]
\end{aligned}
$$

TD: combine the sampling of MC and bootstrapping of DP
- Sample the expected value
- Use a current estimate $V(S_{t+1})$ of the true unknown $v_\pi(S_{t+1})$

DP:
- The expected values are provided by a model
- Use a current estimate $V(S_{t+1})$ of the true unknown $v_\pi(S_{t+1})$

Comparison

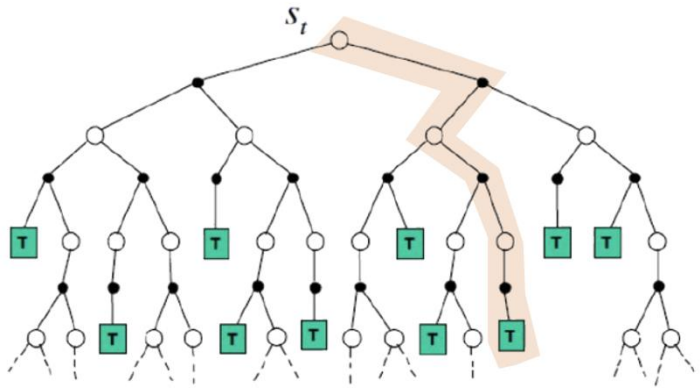


bootstrapping

Dynamic Programming

$V(S_t) \leftarrow \mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1})]$

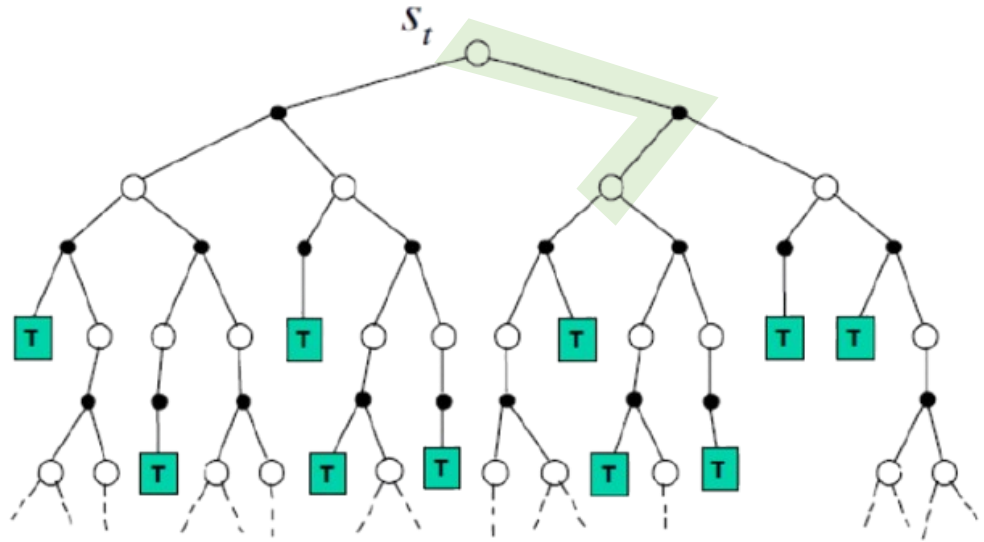
$= \sum_a \pi(a|S_t) \sum_{s',r} p(s', r|S_t, a)[r + \gamma V(s')]$



sampling

Monte-Carlo Methods

$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$ (every visit)



Temporal Difference (TD (0))

$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

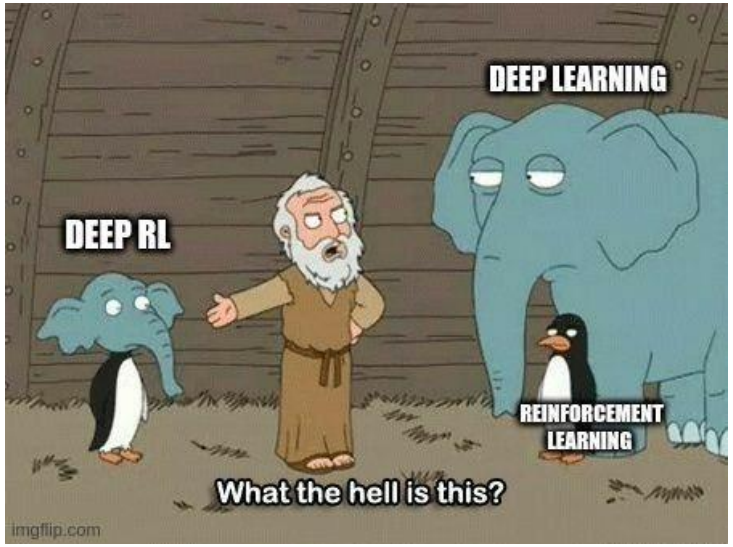
bootstrapping

+

sampling

Bias v.s. Variance

- MC has high variance, zero bias
  - Good convergence properties
  - Not very sensitive to initial value
  - Very simple to understand and use



- TD has low variance, some bias
  - Usually more efficient than MC
  - TD(0) converges to $V^{\pi}(s)$ (but not always with function approximation)
  - More sensitive to initial value

SARSA: On-Policy TD Control

S, A, R, S', A'

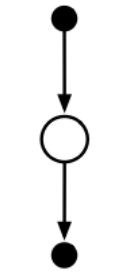
the name of SARSA comes from the transition from state S to S'

- Idea:
  - Continually estimate Q^π for the behavior policy π
  - At the same time change π toward greediness w.r.t. Q^π
- Algorithm:

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal



Sarsa

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on the step sizes, as long as *all state-action pairs are visited an infinite number of times and the policy converges in the limit to the greedy policy*

  - All state-action pairs are explored infinitely many times:
    $$\lim_{k\to\infty} N_k(s,a) = \infty$$
  - The policy converges in the limit to the greedy policy:
    $$\lim_{k\to\infty} \pi_k(a|s) = \mathbb{I}(a = \text{argmax}_{a'\in A} Q_k(s,a'))$$
    - For example, ϵ-greedy is GLIE (greedy in the limit) policies by setting $\epsilon = \frac{1}{t}$ (ϵ reduces to 0)

Q-learning: Off-Policy TD Control

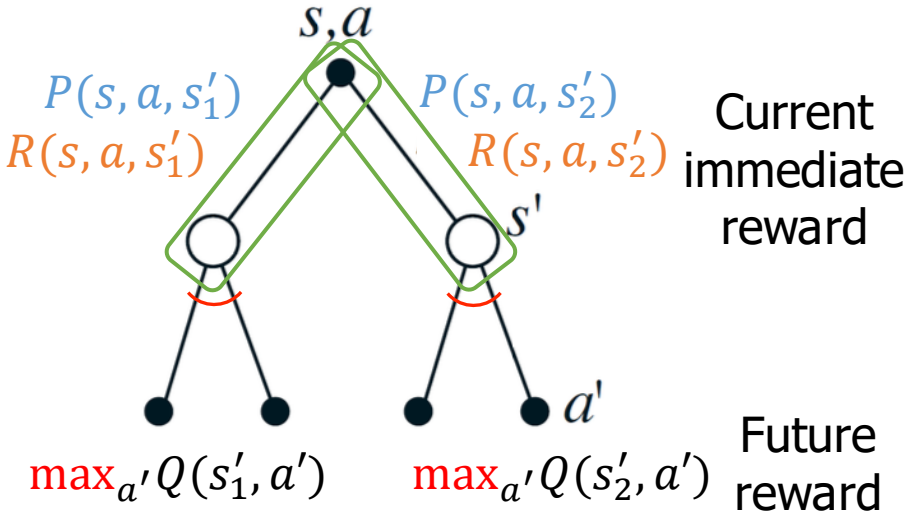
- Bellman optimality equation:

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$

- Q-value iteration: $Q_{k+1}(s,a) \leftarrow \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a')\right]$

⇩ Rewrite as expectation

$$Q_{k+1} \leftarrow \mathbb{E}_{s' \sim P(s'|s,a)}[R(s,a,s') + \gamma \max_{a'} Q_k(s',a')]$$



Current immediate reward

Future reward

- Problem: cannot compute the update without the knowledge of P and R
- Idea: use actual samples to approximate the expectation
  - ↳ ~ drawn from P

Q-learning: Off-Policy TD Control

- Algorithm:

Q-learning: **sampled-based** Q-value iteration

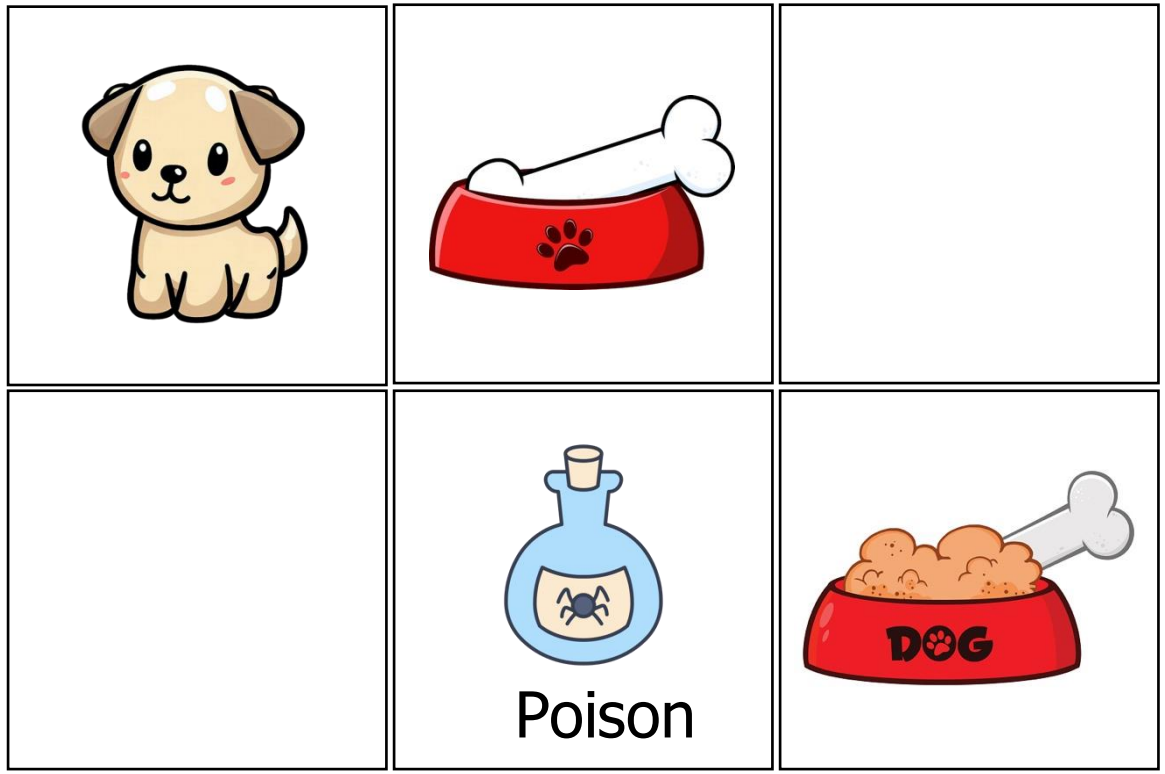
- Learn $Q(s,a)$ as you go
  - ➢ Receive a sample (s,a,r,s')
  - ➢ Compute the current sample: sample $= R(s,a,s') + \gamma \max_{a'} Q(s',a')$

    immediate reward; future returns
  - ➢ Update to $Q(s,a)$ by **running average**: $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha$ sample

    learning rate; new estimate; old estimate; sampled estimated; weighted combination
  - ✓ Effect: Move values toward value of whatever successor occurs
  - ✓ Theoretical guarantee: Convergence

Q-learning: Off-Policy TD Control

- Example:



Reward
- +1: eat the small bone
- +10: eat the animal food with the bone
- -10: eat the poison
- 0: otherwise



Goal: eat the animal food with the bone while avoiding the poison

- Termination: if the dog eats the poison/animal food with the bone, or it spends more than 5 steps

Q-learning: Off-Policy TD Control

- Example:

Initialize the Q-table

Actions

States

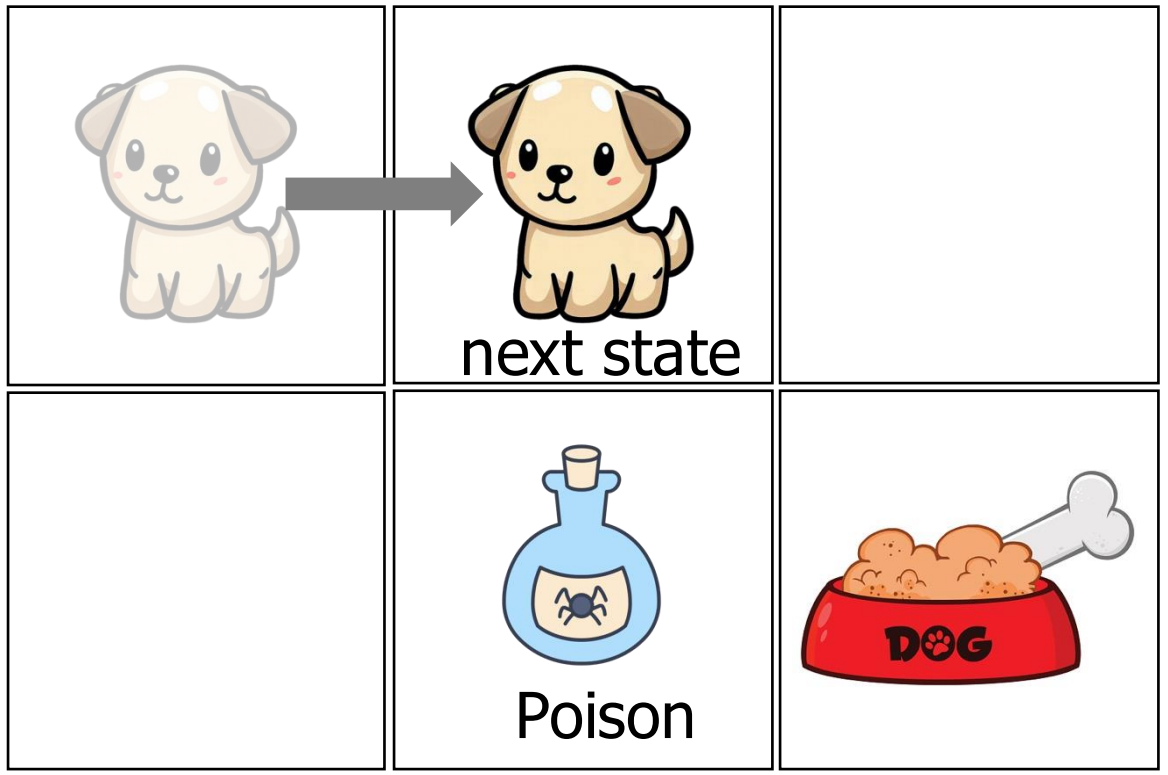
	↑	↓	←	→
(0, 0)	0	0	0	0
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-learning: Off-Policy TD Control

- Example:
  - Choose action a based on a lookup of the Q-table
  - Obtain the next state and reward

	↑	↓	←	→
(0, 0)	0	0	0	0
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-table



Environment

Q-learning: Off-Policy TD Control

- Example:

Update the Q-value

Learning rate

$$Q(s,a) \leftarrow Q(s,a) + \alpha(R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

New Q-value estimation | Previous Q-value estimation | Immediate reward | Discounted estimated optimal next Q-value | Previous Q-value estimation

Target value

$$Q(s=(0,0), a=\rightarrow) = 0 + 0.1 \times [1 + 0.99 \times 0 - 0] = 0.1 \quad (2)$$

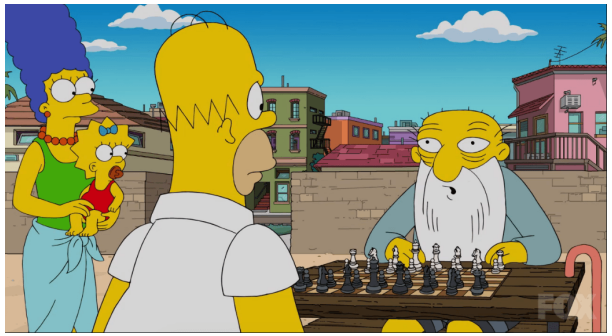
- learning rate $\alpha = 0.1$
- discount factor $\gamma = 0.99$

Q-learning: Off-Policy TD Control

- Amazing result: Q-learning converges to optimal policy -- even if you are acting sub-optimally!
- This is called off-policy learning

Behavior Policy: policy used to navigate in an environment

Target Policy: policy used to optimize decision making



On-policy: behavior policy should be the same as the target policy



Off-policy: behavior policy can be different from target policy

Q-learning: Off-Policy TD Control

- Amazing result: Q-learning converges to optimal policy -- even if you are acting sub-optimally!
- This is called off-policy learning
- Caveats:
  - You have to explore enough
  - You have to eventually make the learning rate small enough
  - … but not decrease it too quickly



Technical requirements:

❑ All states and actions are visited infinitely often
- Basically, in the limit, it does not matter how you select actions (!)

❑ Learning rate schedule such that for all state and action pairs (s, a):

$$\sum_{t=0}^{\infty} \alpha_t(s,a) = \infty \qquad \sum_{t=0}^{\infty} \alpha_t^2(s,a) < \infty$$

Q-learning: Off-Policy TD Control

Model-free Does not require the knowledge of the model

Convergence
- Every (s, a) is visited infinitely often
- Decaying learning rates condition

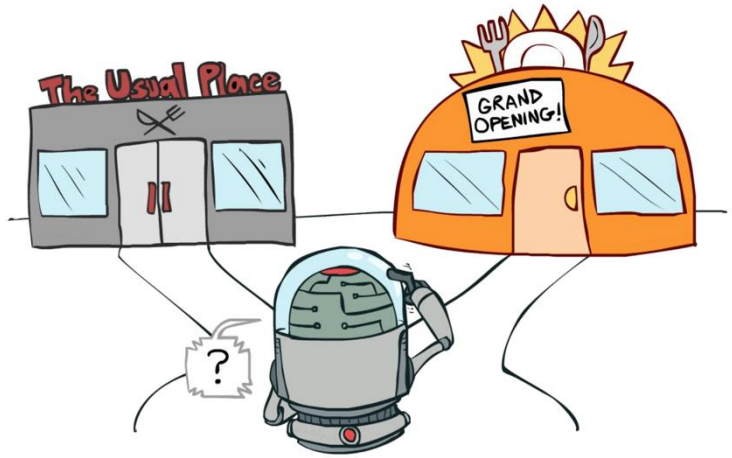
- Does not matter how actions are seletced
- **Exploration/exploitation strategy**
  - ϵ-greedy

    w.p. ϵ: act randomly (**exploration**)

    w.p. $1 - \epsilon$: act according to the current greedy policy (**exploitation**)



(Can lower ϵ over time)



Off-policy Sample-efficient

Checklist

- **Markov Decision Process (MDP)**
  - Core components
  - Understanding how MDPs formalize sequential decision problems
  - Bellman equations as recursive relationships between value functions
- **Value Functions and Optimality**
  - State-value functions and action-value functions
  - Optimal policy and its relationship to optimal value functions
  - Recognizing different Bellman operators from their mathematical forms
- **Dynamic Programming**
  - Value Iteration: update equation, convergence properties, computational complexity
  - Policy Iteration: policy evaluation and policy improvement steps, computational complexity
  - How discount factor affects optimal policies

Checklist

- **Monte Carlo vs. Temporal Difference Learning**
  - Monte Carlo methods: learning from complete episodes, unbiased but high variance
  - TD learning: bootstrapping from incomplete episodes, biased but lower variance
  - TD error calculation
- **Q-learning and SARSA**
  - Q-learning as an off-policy control algorithm
  - SARSA as an on-policy alternative
  - Update equations and implementation details
  - How exploration policies affect learning
- **Exploration Strategies**
  - ε-greedy approach to balancing exploration and exploitation
  - GLIE (Greedy in the Limit with Infinite Exploration) conditions
  - Impact of exploration parameters on learning performance

Checklist

- **On-Policy vs. Off-Policy Learning**
  - Fundamental differences between learning approaches
  - Advantages and limitations of each approach
- **Practical Problem-Solving**
  - Grid world navigation problems
  - Computing optimal policies for simple environments
  - Step-by-step value updates in tabular settings
  - Feature-based representation for more complex problems

香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

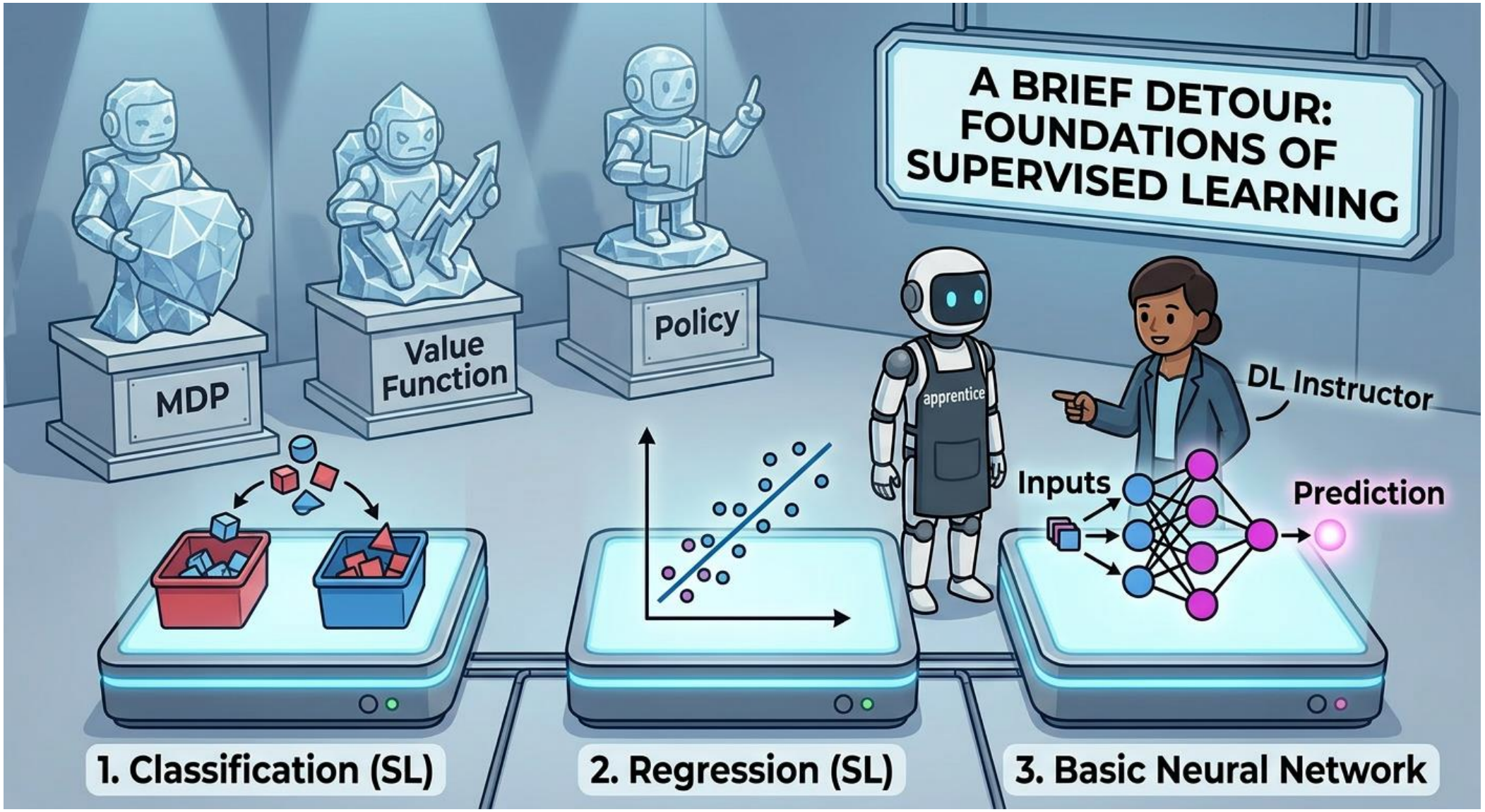
ELEC6910J
Deep Reinforcement Learning

Lecture 14. Supervised Learning: Regression & Classification

Course Instructor: Ling PAN
Department of Electronic and Computer Engineering
Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188, CMU 10-403, UCL COMPM050

Part 2: Supervised Learning & Deep Neural Networks



Q1: How do regression and classification work?

- Simplest setting: fixed data, perfect labels

→ Regression & Classification

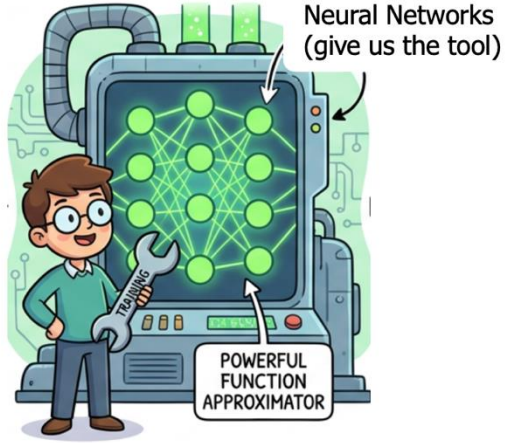


Supervised Learning
(gives us the language)

Q2: What are neural networks and how do we train them?

- What if linear models aren't expressive enough?

→ Neural Networks



Q3: How do we adapt all of this to RL? (Deep RL)

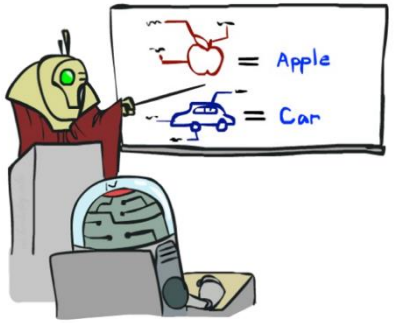
- Now bring NN back into RL

→ RL with Function Approximation



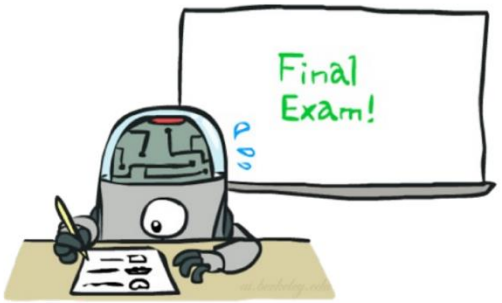
Supervised Learning

- Big idea: ML algorithms learn patterns between features and labels from data
  - You do not have to reason about the data yourself
  - You are given training data: lots of example datapoints and their actual labels





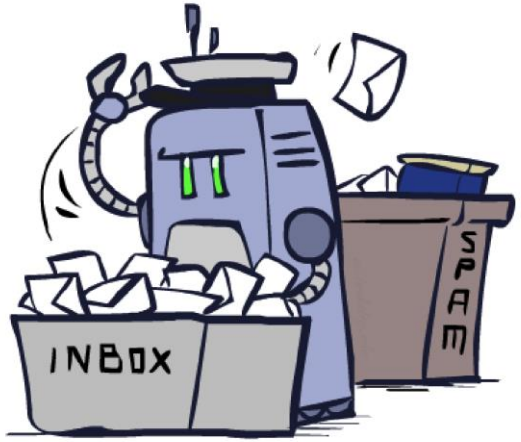
Training: Learn patterns from labeled data, and periodically test how well you're doing

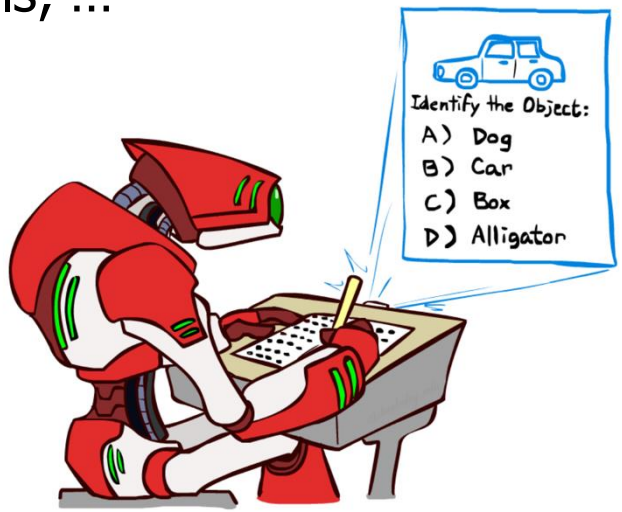


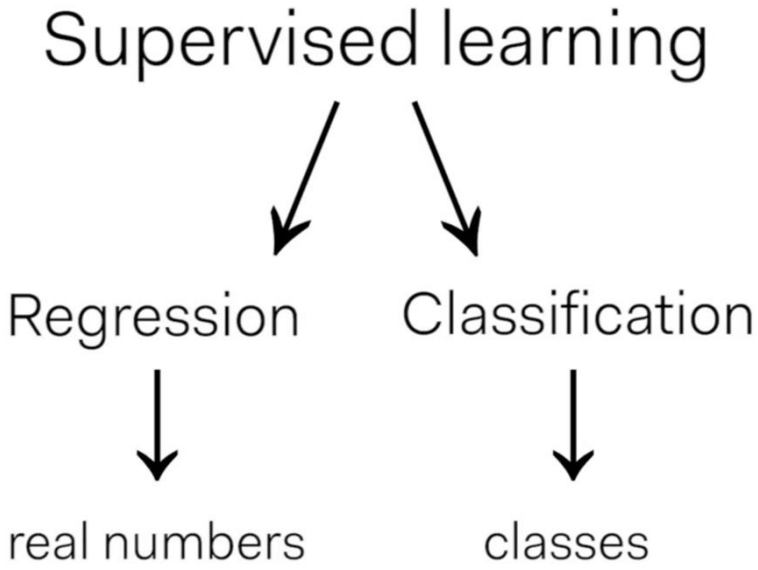
Eventually, use your algorithm to predict labels for unlabeled data

Supervised Learning

- Classification: given inputs x, predict labels (classes) y
  - Predicts if something belongs to a particular category, and it could be binary or multiclass classification
  - Example: spammed emails, …



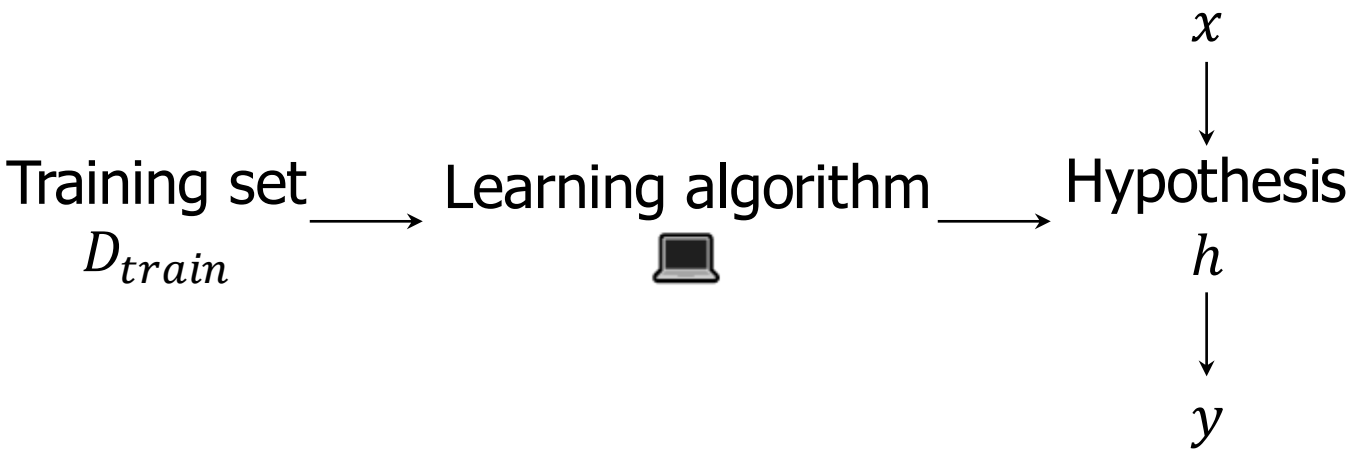




- Regression: predicts a numeric value
  - Example: future house price prediction, ride time estimation, …

Supervised Learning

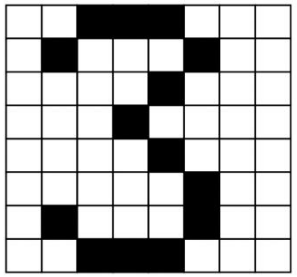
- Goal: given given a training set, to learn a hypothesis function h: $\mathcal{X} \rightarrow \mathcal{Y}$ so that $h(x)$ is a good predictor for the corresponding value of y (a good way to label new features)



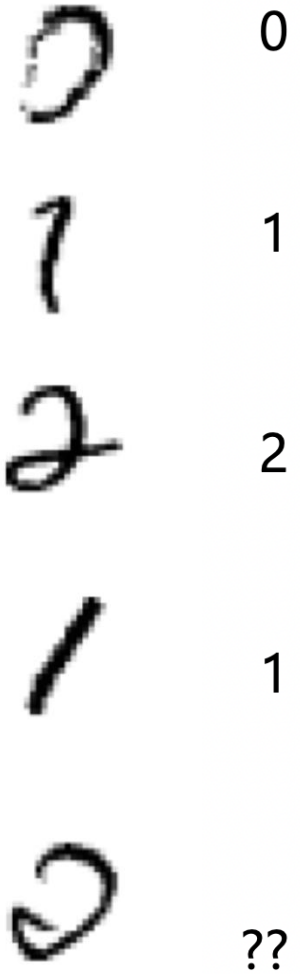
  - Regression: when the target variable that we're trying to predict is continuous (e.g., the housing example)
  - Classification: when y can take on only a small number of discrete values (e.g., digit recognition example)

Supervised Learning: Classification

- Example: Digit recognition
  - Input: images
  - Output: a digit 0-9

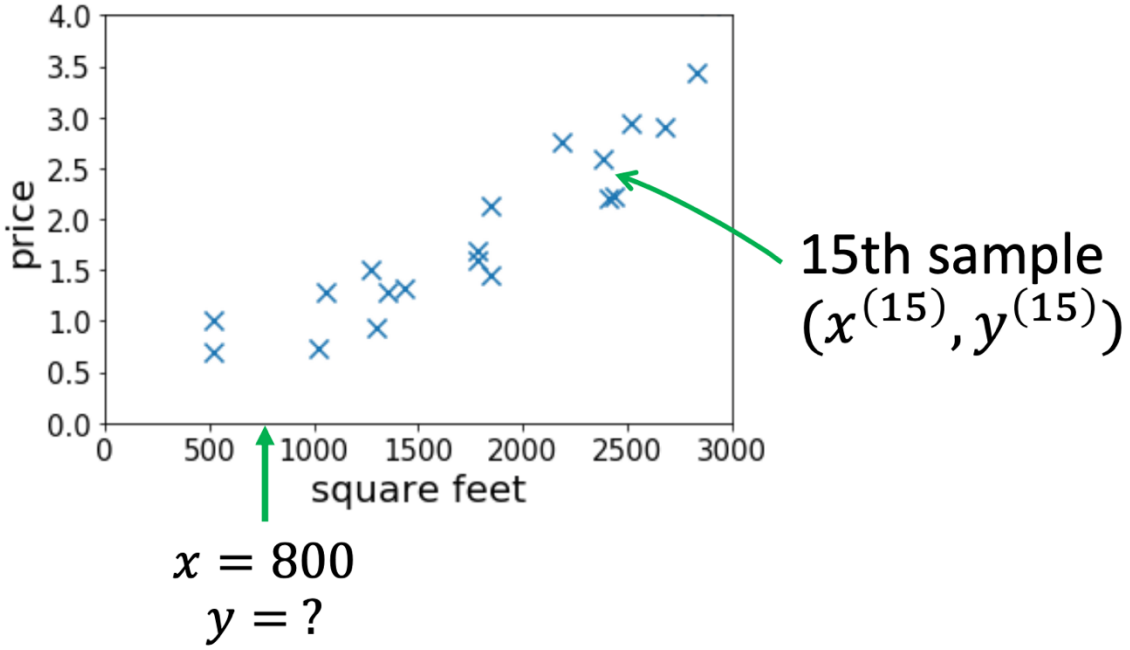


- Setup:
  - Get a large collection of example images, each labeled with a digit
  - Note: someone has to hand label all this data!
  - Want to learn to predict labels of new, future digit images
- Features: The attributes used to make the digit prediction
  - Pixels: (6,8)=ON
  - Shape Patterns: NumComponents, AspectRatio, NumLoops
  - …



Supervised Learning: Regression

- Given: a dataset that contains n samples $((x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)}))$
- Task: if a residence has x square feet, predict its price?



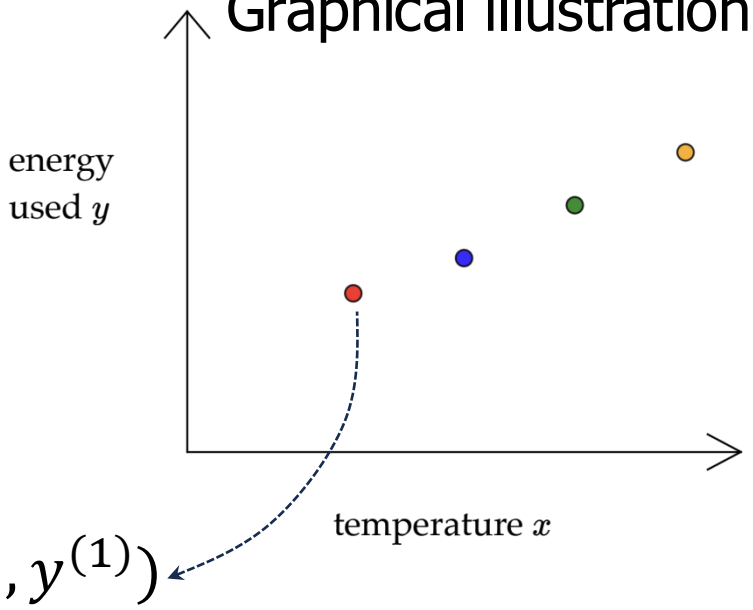
- $x^{(i)}$: input features, e.g., living area
- $y^{(i)}$: target that we are trying to predict, e.g., price
- $(x^{(i)}, y^{(i)})$: a training example
- $\{(x^{(i)}, y^{(i)}); i = 1, \ldots n\}$: training set
- (i): the superscript notation (has noting to do with exponentiation)
- $\mathcal{X}$: the space of input values, e.g., $\mathcal{X} = \mathbb{R}$
- $\mathcal{Y}$: the space of output values, e.g., $\mathcal{Y} = \mathbb{R}$

Example: Predict a City's Energy Usage

- Go collect some data in various cities

City	Feature	Label
	Temperature	Energy Used
Chicago	90	45
New York	20	32
Boston	35	99
San Diego	18	39

Graphical illustration



Matrix-vector form

$$X = \begin{bmatrix} 90 \\ 20 \\ 35 \\ 18 \end{bmatrix} \in \mathbb{R}^{4\times 1} \qquad Y = \begin{bmatrix} 45 \\ 32 \\ 99 \\ 39 \end{bmatrix} \in \mathbb{R}^{4\times 1}$$

Training data $\mathcal{D}_{\text{train}} := \left\{ \left(x^{(1)}, y^{(1)}\right), \ldots, \left(x^{(4)}, y^{(4)}\right) \right\}$

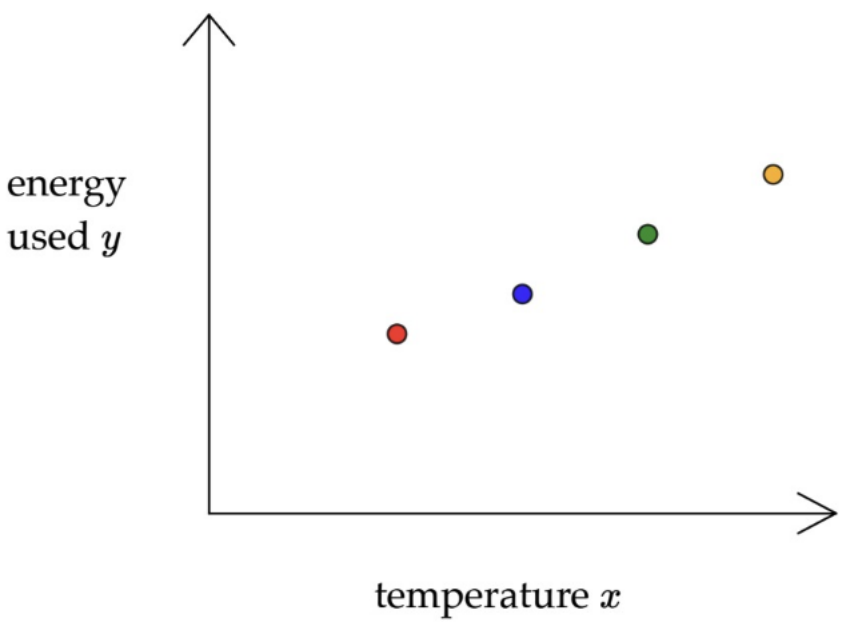
$n = 4, d = 1$

Example: Predict a City's Energy Usage

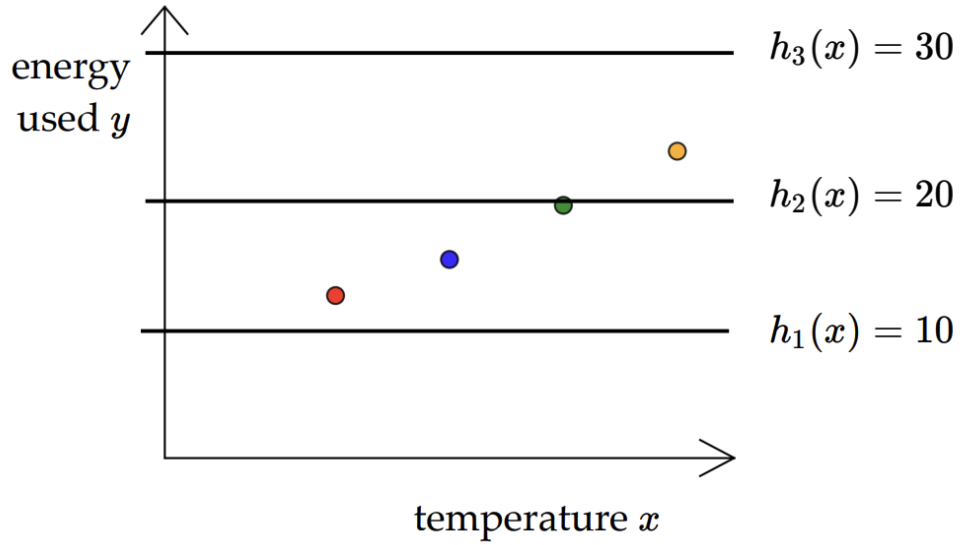
- We need to decide how we are going to represent function/hypotheses h

→ Hypothesis class $\mathcal{H}$: set of h we ask the algorithm to search over

$$\mathcal{D}_{\text{train}} := \left\{\left(x^{(1)}, y^{(1)}\right), \ldots, \left(x^{(4)}, y^{(4)}\right)\right\}$$



- Idea 1: constant functions



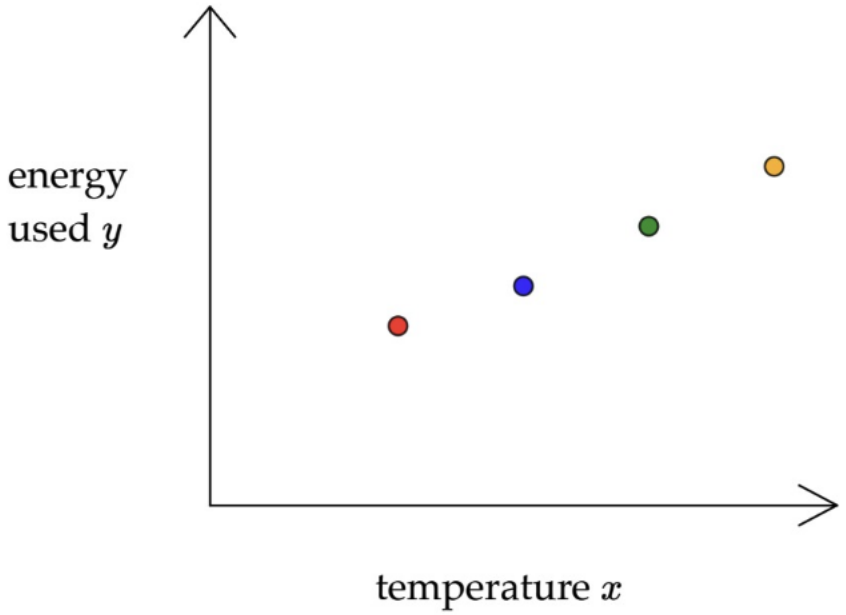
- Not expressive…

Example: Predict a City's Energy Usage

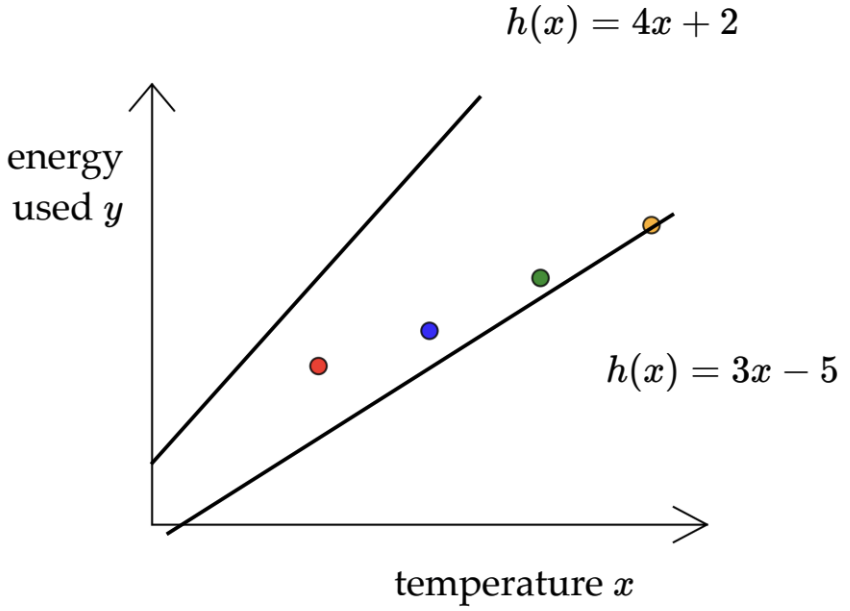
- We need to decide how we are going to represent function/hypotheses h

→ Hypothesis class $\mathcal{H}$: set of h we ask the algorithm to search over

$$\mathcal{D}_{\text{train}} := \left\{\left(x^{(1)}, y^{(1)}\right), \ldots, \left(x^{(4)}, y^{(4)}\right)\right\}$$



- Idea 2: approximate y as a linear function of x



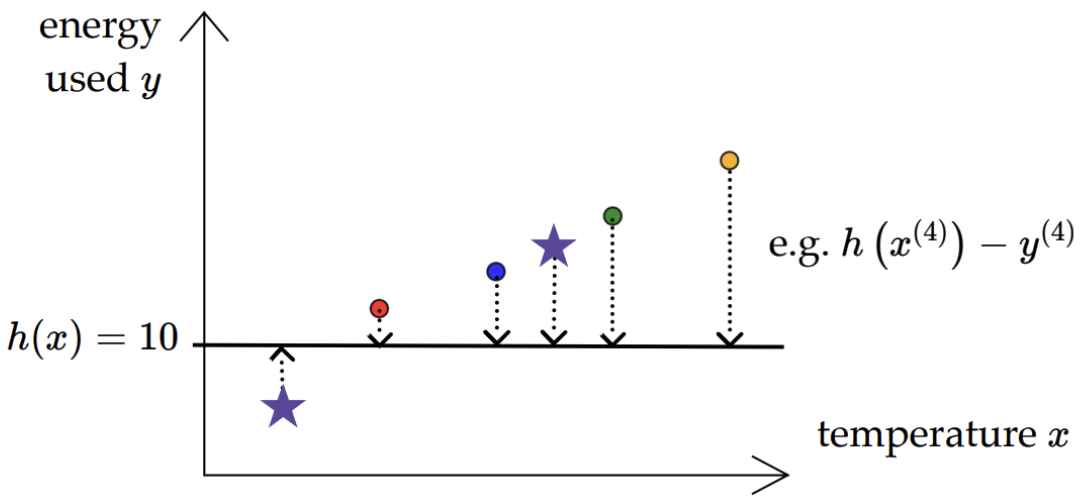
$$h(x) = \theta_1 x + \theta_0$$

- θ_i: the parameters/weights parametrizing the space of linear functions mapping from $\mathcal{X}$ to $\mathcal{Y}$

- More expressive

Example: Predict a City's Energy Usage

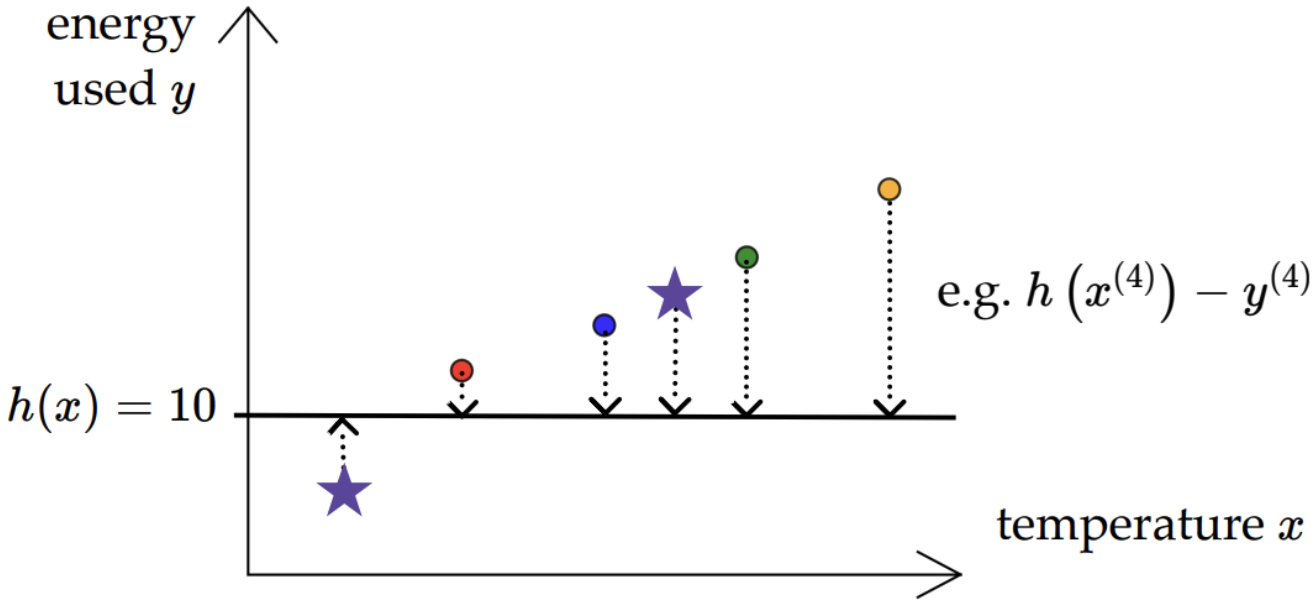
- Given a training set, how do we learn the parameters θ?
- Idea: make $h(x)$ close to y, at least for the training examples we have.
  - Define a cost function that measures, for each value of the θ's, how close the $h(x^{(i)})$'s are to the corresponding $y^{(i)}$'s:



- Question: how well the model is fit to the data?
- Residual for the i-th sample: $\epsilon^{(i)} = h\left(x^{(i)}\right) - y^{(i)}$
  - Positive → we predicted too high
  - Negative → we predicted too low
  - Zero → perfect prediction
- The naïve idea: add up all the errors $\sum_{i=1}^{n} \epsilon^{(i)}$
  - Positive and negative errors cancel out…
  - We need a way to make all errors count as positive

Example: Predict a City's Energy Usage

- Given a training set, how do we learn the parameters θ?
- Idea: make $h(x)$ close to y, at least for the training examples we have.
  - Define a cost function that measures, for each value of the θ's, how close the $h(x^{(i)})$'s are to the corresponding $y^{(i)}$'s:



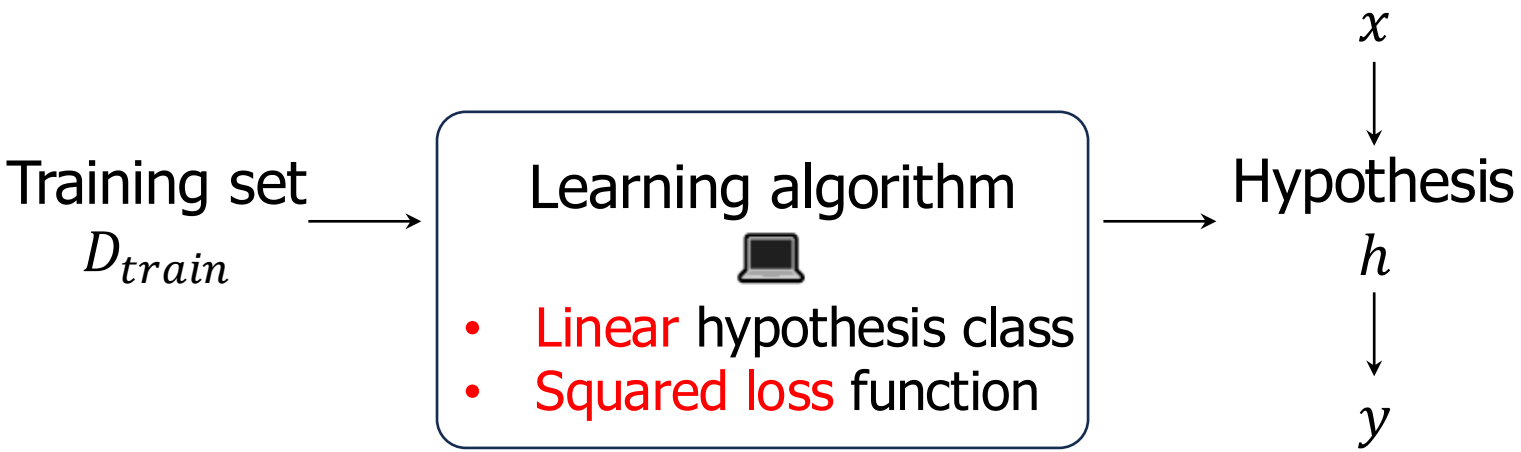
Least-squares cost function

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{n}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$

- Why 1/2? When we take the derivative, the exponent 2 comes down and cancels with ½ (will be discussed later)

→ purely for notational convenience (doesn't change which θ minimizes J).

Supervised Learning: Regression



Example: Predict a City's Energy Usage

- What if there are multiple features?
  - Go collect some data in various cities

City	Features		Label
	Temperature	Population	Energy Used
Chicago	90	7.2	45
New York	20	9.5	32
Boston	35	8.4	99
San Diego	18	4.3	39

Training data

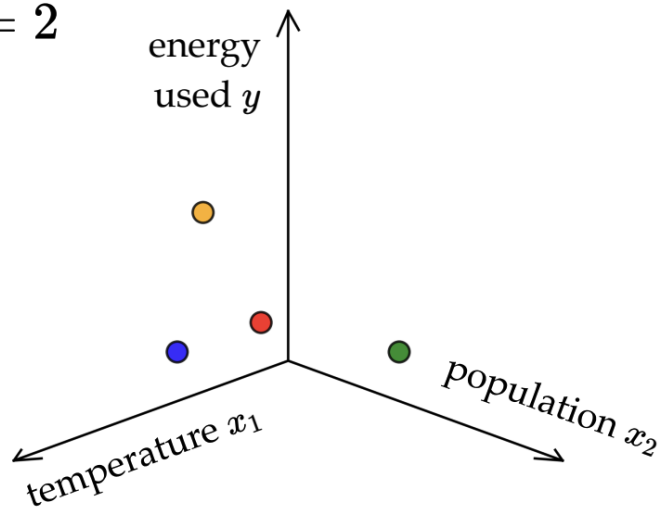
$$\mathcal{D}_{\text{train}} := \left\{ \left(x^{(1)}, y^{(1)}\right), \ldots, \left(x^{(4)}, y^{(4)}\right) \right\}$$

feature vector

label

$$x^{(1)} = \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \end{bmatrix}$$

$n = 4, d = 2$



$$X = \begin{bmatrix} 90 & 7.2 \\ 20 & 9.5 \\ 35 & 8.4 \\ 18 & 4.3 \end{bmatrix} \in \mathbb{R}^{4\times 2} \qquad Y = \begin{bmatrix} 45 \\ 32 \\ 99 \\ 39 \end{bmatrix} \in \mathbb{R}^{4\times 1}$$

Supervised Learning: Regression

- Training data: $\mathcal{D}_{\text{train}} := \left\{\left(x^{(1)}, y^{(1)}\right), \ldots, \left(x^{(n)}, y^{(n)}\right)\right\}$

$$x^{(1)} = \begin{bmatrix} x_0^{(1)} \\ x_1^{(1)} \\ \vdots \\ x_d^{(1)} \end{bmatrix} \in \mathbb{R}^d$$ feature vector

label $y^{(1)} \in \mathbb{R}$

  - n data points, each with $d+1$-dimensional features (since $x_0 = 1$) and scalar label

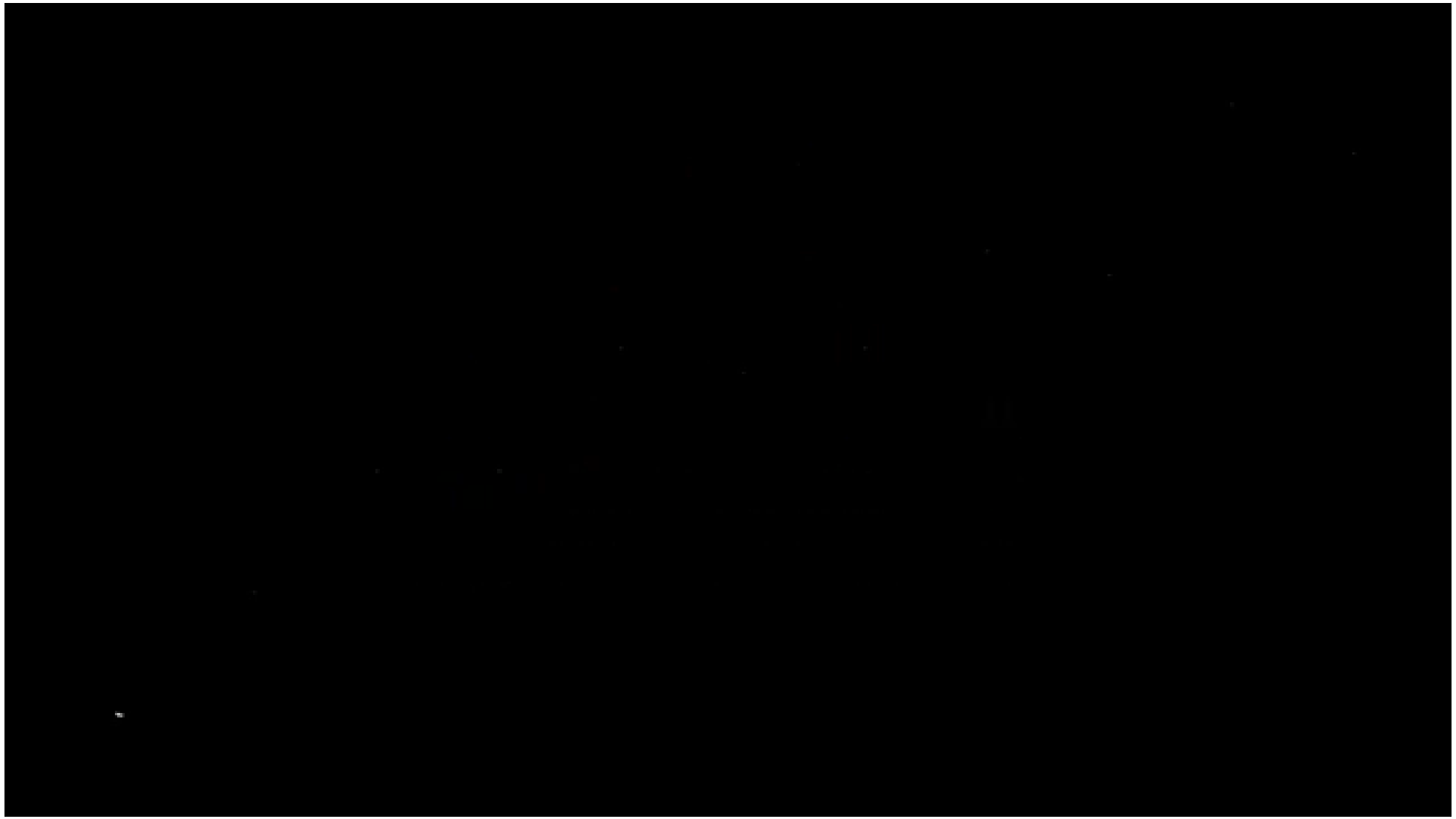
- Hypothesis function: $h(x) = \sum_{i=0}^{d} \theta_i x_i = \theta^T x = [\theta_0 \quad \theta_1 \quad \ldots \quad \theta_d] \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$

  - Introduce the convention of letting $x_0 = 1$ (the intercept term)
  - θ: parameter vector
  - x: input features vector

- Squared loss function: $\mathcal{L}\left(h\left(x^{(i)}\right), y^{(i)}\right) = (\theta^T x^{(i)} - y^{(i)})^2$

- Goal: we want to learn θ such that $h(x) \approx y$ But how?

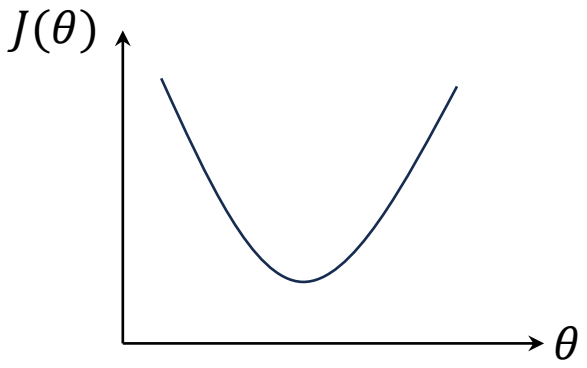
Supervised Learning: Regression



Normal Equations: Starting from the One-Variable Case

- Suppose we have just one feature (no intercept), so our model is: $h(x) = \theta x$
- We have n training examples $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$
- The loss function is: $J(\theta) = \frac{1}{2}\sum_{i=1}^{n}(\theta x^{(i)} - y^{(i)})^2$
- Goal: find the value of θ that minimizes $J(\theta)$

- Step 1. Expand $J(\theta)$: $J(\theta) = \frac{1}{2}\underbrace{\left(\sum_{i=1}^{n}\left(x^{(i)}\right)^2\right)}_{a>0}\theta^2 - \underbrace{\left(\sum_{i=1}^{n} x^{(i)}y^{(i)}\right)}_{b}\theta + \frac{1}{2}\sum_{i=1}^{n}\left(y^{(i)}\right)^2$



An upward-opening parabola
→ Has exactly one minimum

Normal Equations: Starting from the One-Variable Case

- Suppose we have just one feature (no intercept), so our model is: $h(x) = \theta x$
- We have n training examples $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$
- The loss function is: $J(\theta) = \frac{1}{2}\sum_{i=1}^{n}(\theta x^{(i)} - y^{(i)})^2$
- Goal: find the value of θ that minimizes $J(\theta)$

- Step 2. Take the derivative and set it to 0
  - ➢ Review: from calculus, the minimum of a parabola is where the derivative equals zero.

Derivative: $\frac{d}{d\theta}\frac{1}{2}\left(\theta x^{(i)} - y^{(i)}\right)^2 = \left(\theta x^{(i)} - y^{(i)}\right) \cdot x^{(i)} \rightarrow \frac{dJ}{d\theta} = \sum_{i=1}^{n}\left(\theta x^{(i)} - y^{(i)}\right) \cdot x^{(i)}$

Set the derivative to 0: $\sum_{i=1}^{n}\left(\theta x^{(i)} - y^{(i)}\right) \cdot x^{(i)} = 0 \rightarrow \theta\sum_{i=1}^{n}\left(x^{(i)}\right)^2 = \sum_{i=1}^{n} x^{(i)}y^{(i)}$

$\rightarrow \theta^* = \frac{\sum_{i=1}^{n} x^{(i)}y^{(i)}}{\sum_{i=1}^{n}\left(x^{(i)}\right)^2}$

A closed-form/ analytical solution: we have a direct formula for the optimal θ

- No iteration, no guessing an initial value → Just plug in the data & compute

Normal Equations: Starting from the One-Variable Case

- Suppose we have just one feature (no intercept), so our model is: $h(x) = \theta x$
- We have n training examples $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$
- The loss function is: $J(\theta) = \frac{1}{2}\sum_{i=1}^{n}(\theta x^{(i)} - y^{(i)})^2$
- Goal: find the value of θ that minimizes $J(\theta)$
- Step 2. Take the derivative and set it to 0
  - Review: from calculus, the minimum of a parabola is where the derivative equals zero.

Derivative: $\frac{d}{d\theta}\frac{1}{2}(\theta x^{(i)} - y^{(i)})^2 = (\theta x^{(i)} - y^{(i)}) \cdot x^{(i)} \Rightarrow \frac{dJ}{d\theta} = \sum_{i=1}^{n}(\theta x^{(i)} - y^{(i)}) \cdot x^{(i)}$

Set the derivative to 0: $\sum_{i=1}^{n}(\theta x^{(i)} - y^{(i)}) \cdot x^{(i)} = 0 \Rightarrow \theta\sum_{i=1}^{n}(x^{(i)})^2 = \sum_{i=1}^{n} x^{(i)} y^{(i)}$

$\Rightarrow \theta^* = \frac{\sum_{i=1}^{n} x^{(i)} y^{(i)}}{\sum_{i=1}^{n}(x^{(i)})^2}$

A closed-form/ analytical solution: we have a direct formula for the optimal θ

- No iteration, no guessing an initial value → Just plug in the data & compute

What if our model has multiple parameters?

$$h(x) = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d$$

- Take a partial derivative with respect to each θ_j, set them all to 0
- Solve a system of d+1 equations with d+1 unknowns.
- Writing this out by hand gets messy very quickly…
  - Matrix notation lets us organize all of these equations into one clean expression 😊

Normal Equations

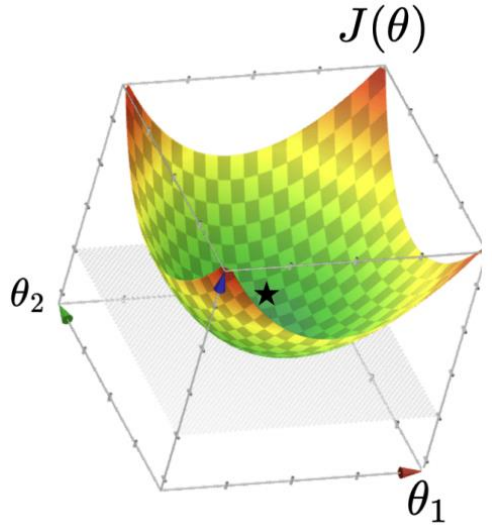
- Summary:
  - Step 1: Write the training error $J(\theta)$ in scalar form
  - Step 2: Rearrange into matrix-vector form
  - Step 3: Set the gradient to $\nabla_\theta J(\theta)$ zero
  - Step 4: Solve for the optimal parameters $\theta^* = \left(X^\top X\right)^{-1} X^\top Y$



Normal Equations

- Exercise: predict a city's energy usage

City	Features		Label
	Temp (°F)	Pop (M)	Energy (kWh)
Chicago	90	7.2	45
New York	20	9.5	32
Boston	35	8.4	99
San Diego	18	4.3	39



Quadratic function that looks like a bowl

- Our model: $\hat{y} = \theta_1 x_1 + \theta_2 x_2$ (no bias/intercept term θ_0 in this example)
- We want to find θ_1 and θ_2 (θ^*)that minimize the prediction error across all 4 cities.

Step 1: Write the training error $J(\theta)$ in scalar form

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{n}(h_\theta(x^{(i)}) - y^{(i)})^2$$

$$J(\theta) = \frac{1}{2}\left[(\theta_1 \cdot 90 + \theta_2 \cdot 7.2 - 45)^2 + (\theta_1 \cdot 20 + \theta_2 \cdot 9.5 - 32)^2 + (\theta_1 \cdot 35 + \theta_2 \cdot 8.4 - 99)^2 + (\theta_1 \cdot 18 + \theta_2 \cdot 4.3 - 39)^2\right]$$

Normal Equations

- Exercise: predict a city's energy usage
  - Writing out 4 terms is manageable. But what if we had 10,000 cities?

→ We need a compact notation: Step 2: rewrite in matrix-vector form

City	Features		Label
	Temp (°F)	Pop (M)	Energy (kWh)
Chicago	90	7.2	45
New York	20	9.5	32
Boston	35	8.4	99
San Diego	18	4.3	39

$$X = \begin{bmatrix} 90 & 7.2 \\ 20 & 9.5 \\ 35 & 8.4 \\ 18 & 4.3 \end{bmatrix} \in \mathbb{R}^{4\times 2}, \quad Y = \begin{bmatrix} 45 \\ 32 \\ 99 \\ 39 \end{bmatrix} \in \mathbb{R}^{4\times 1}, \quad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} \in \mathbb{R}^{2\times 1}$$

- Step 3-4: Set the gradient $\nabla_\theta J(\theta)$ to 0 to solve for the optimal parameters:

$$\theta^* = \left(X^\top X\right)^{-1} X^\top Y$$

  - Compute $X^T Y$
  - Compute $X^T X$ and its inverse $(X^T X)^{-1}$
  - Find the answer

Normal Equations

- The beauty of $\theta^* = (X^\top X)^{-1} X^\top Y$

Pros:

- When θ^* is well defined, it's the unique minimizer of $J(\theta)$
- Closed-form solution, does not feel like "training"

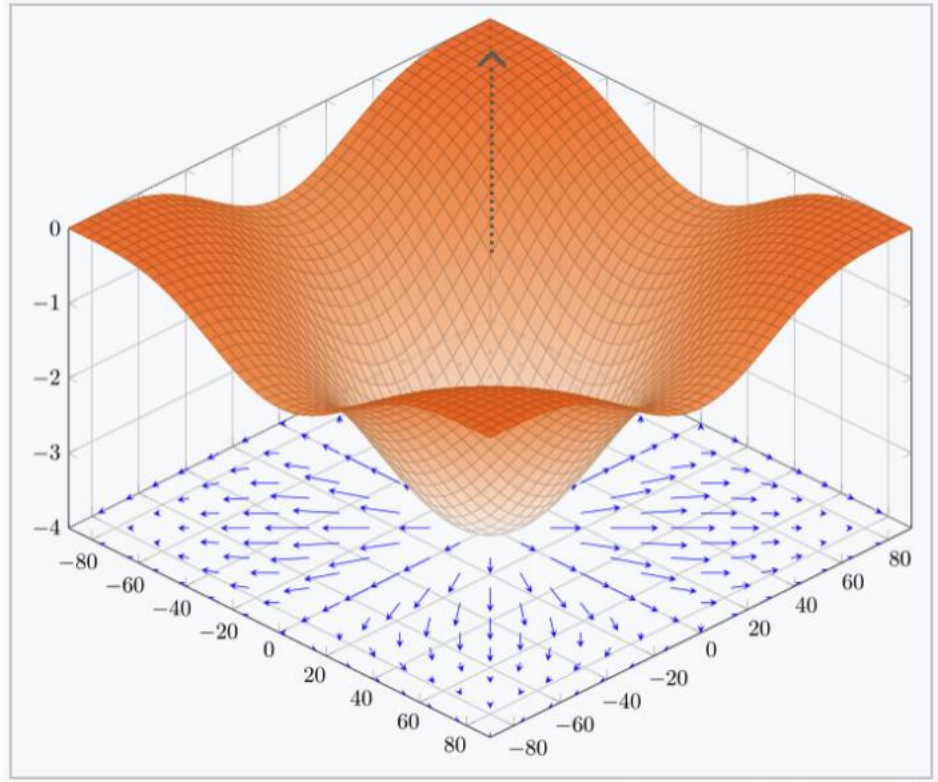
Cons:

- The matrix $X^T X$ may not be invertible
- It gets computationally expensive when d is large (matrix inversion)
- Very rare case where we get a general and clean solution with nice theoretical guarantee

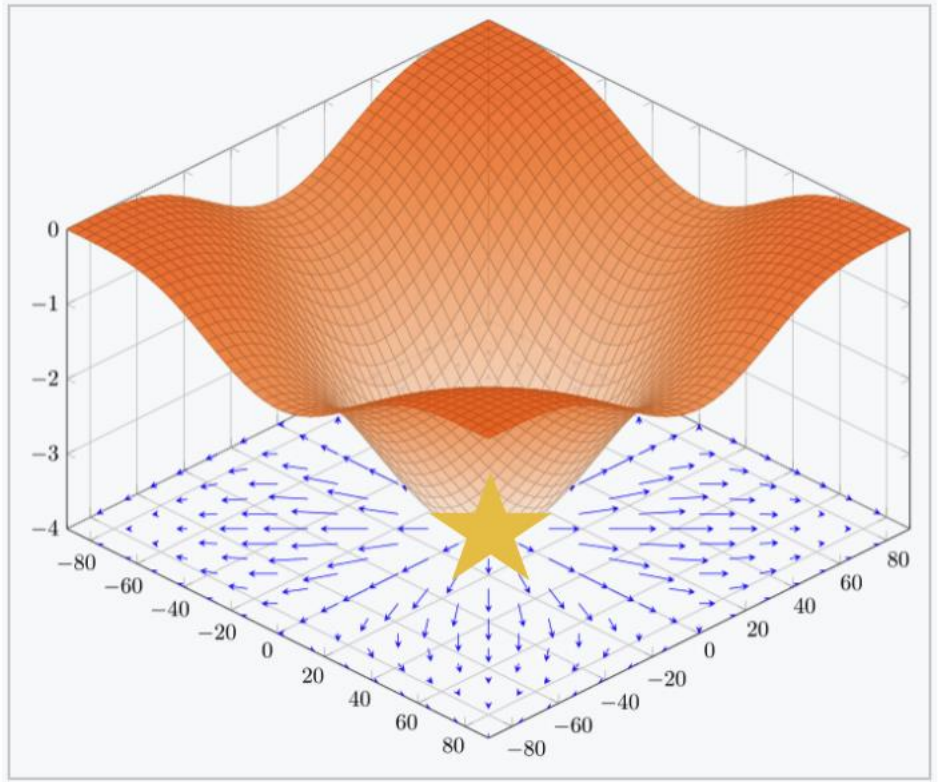
$\rightarrow$ Need a more efficient and general algorithm to train our ML system: gradient descent methods

Gradient Descent

- Gradient:
  - The gradient points in the direction of the (steepest) increase in the function value.
  - The gradient at the function minimizer is necessarily zero.



The gradient of the function $f(x,y) = -(\cos^2 x + \cos^2 y)^2$



The gradient of the function $f(x,y) = -(\cos^2 x + \cos^2 y)^2$

Gradient Descent

- Goal: We want to choose θ so as to minimize J(θ), i.e., $\theta = \underset{\theta}{\operatorname{argmin}}\, J(\theta)$

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{n}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2, \text{ where } h_\theta(x) = \sum_{j=0}^{d}\theta_j x_j = x^T\theta$$

- Intuition: walking downhill blindfolded
  - Feel the slope under your feet (compute the gradient)
  - Take a small step in the downhill direction (update the parameters)
  - Repeat 1–2 until you reach the valley floor (convergence)

Gradient Descent

- Goal: We want to choose θ so as to minimize J(θ), i.e., $\theta = \underset{\theta}{\operatorname{argmin}}\, J(\theta)$

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{n}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2, \text{ where } h_\theta(x) = \sum_{j=0}^{d}\theta_j x_j = x^T\theta$$

- Idea: use a search algorithm that
  - Starts with some initial guess for θ
  - Repeatedly changes θ to make J(θ) smaller
  - Until hopefully we converge to a value of θ that minimizes J(θ)

→ Gradient descent algorithm

Gradient Descent

- Starts with some initial θ
- Repeatedly performs the update:

Partial derivative of J with respect to θ_j
- If I nudge θ_j a tiny bit (keeping all other parameters fixed), how much does J change?
  - Positive = J goes up
  - Negative = J goes down

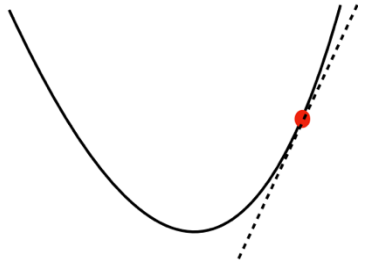
Learning rate
- How big is each step?

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

-: The partial derivative tells us the direction that increases J; we want to decrease J, so we negate it

Current value of the j-the parameter
- Where you are standing right now

- Simultaneously performed for all values of j = 0, . . . , n
- Effect: repeatedly takes a step in the direction of steepest decrease of J



Gradient Descent

- Starts with some initial θ
- Repeatedly performs the update:

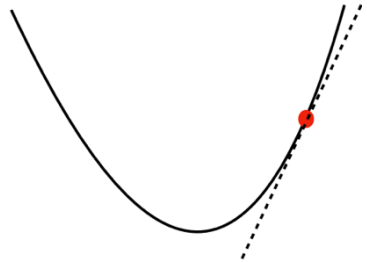
Partial derivative of J with respect to θ_j

- If I nudge θ_j a tiny bit (keeping all other parameters fixed), how much does J change?
  - Positive = J goes up
  - Negative = J goes down

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

-: The partial derivative tells us the direction that increases J; we want to decrease J, so we negate it

- $\frac{\partial}{\partial \theta_j} J(\theta) > 0$: increasing θ_j makes J goes up (bad) → we should decrease θ_j
  - $\theta_j = \theta_j - \alpha \cdot$ positive → θ_j gets smaller (J was going up, so pull back)
- $\frac{\partial}{\partial \theta_j} J(\theta) < 0$: increasing θ_j makes J goes down (good) → we should increase θ_j
  - $\theta_j = \theta_j - \alpha \cdot$ negative → θ_j gets larger (J was going down, so keep going)
- $\frac{\partial}{\partial \theta_j} J(\theta) = 0$:
  - $\theta_j = \theta_j$ → no change (already at the bottom!)



➢ The minus sign automatically pushes θ_j in the direction that reduces J.

Gradient Descent

- What exactly is this slope?
  - Start with a single training example: assume we have only one training example (x, y)

$$J(\theta) = \tfrac{1}{2}\left(h_\theta(x) - y\right)^2$$

  - Take the partial derivative with respect to θ_i:

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} \left(h_\theta(x) - y\right)^2 \\
&= 2 \cdot \frac{1}{2} \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial \theta_j} \left(h_\theta(x) - y\right) \\
&= \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{n} \theta_i x_i - y \right) \\
&= \left(h_\theta(x) - y\right) x_j
\end{aligned}$$

chain rule

Gradient Descent

- What exactly is this slope?

→ For a single training example, this gives the update rule:

$$\theta_j := \theta_j + \alpha \underbrace{\left(y^{(i)} - h_\theta(x^{(i)})\right)}_{\text{prediction error}} x_j^{(i)}.$$

- The magnitude of the update is proportional to the error term $\left(y^{(i)} - h_\theta(x^{(i)})\right)$
  - Our prediction nearly matches the actual value of $y^{(i)}$: there is little need to change the parameters
  - Our prediction has a large error (i.e., if it is very far from $y^{(i)}$): a larger change to the parameters will be made
- Generalize to a training set of multiple training examples:
  - ❑ Derivative of sum = sum of derivatives

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 \rightarrow \frac{\partial}{\partial\theta_j}J(\theta) = \sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)\cdot x_j^{(i)}$$

Gradient Descent

- The complete algorithm (batch gradient descent)

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)} \qquad \text{(for every } j\text{)}.$$

}

- Looks at every example in the entire training set on every step
  - Theoretical guarantee: While gradient descent can be susceptible to local minima in general, the optimization problem we have posed here for linear regression has only one global, and no other local, optima; thus gradient descent always converges (assuming the learning rate α is not too large) to the global minimum

Gradient Descent

hyperparameters — initial guess of parameters — precision

Algorithm 1 Gradient Descent($\theta_{\text{init}}, \eta, J, \epsilon$)

1: Initialize $\theta^{(0)} = \theta_{\text{init}}$
2: Initialize $t = 0$
3: **repeat**
4: $\quad t = t + 1$
5: $\quad \theta^{(t)} = \theta^{(t-1)} - \eta \nabla_\theta J(\theta^{(t-1)})$
6: **until** $\left| J(\theta^{(t)}) - J(\theta^{(t-1)}) \right| < \epsilon$
7: **return** $\theta^{(t)}$

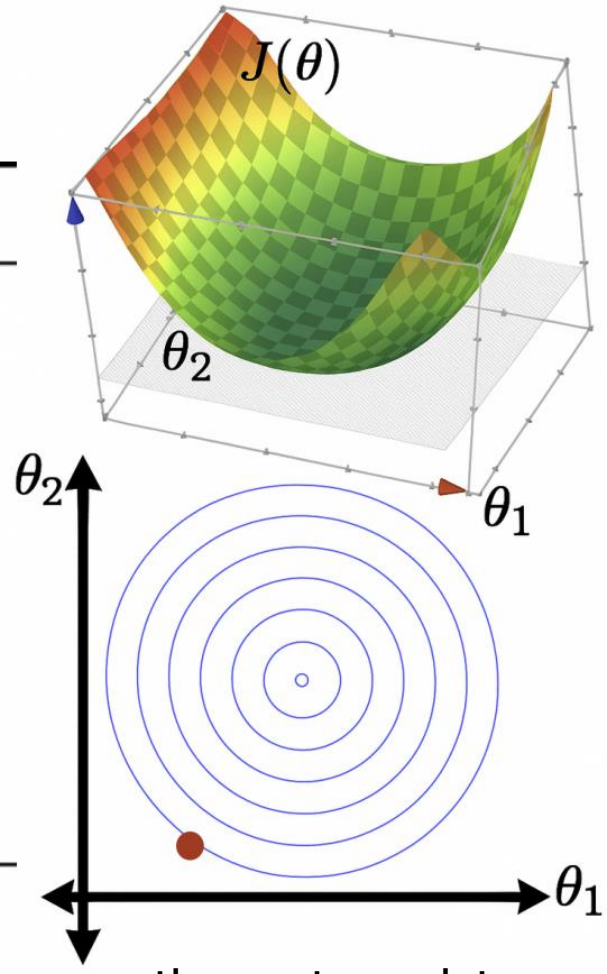
learning rate

Gradient Descent

Algorithm 1 Gradient Descent$(\theta_{\text{init}}, \eta, J, \epsilon)$

1: Initialize $\theta^{(0)} = \theta_{\text{init}}$
2: Initialize $t = 0$ ← iteration counter
3: **repeat**
4: $\quad t = t + 1$
5: $\quad \theta^{(t)} = \theta^{(t-1)} - \eta \nabla_\theta J(\theta^{(t-1)})$
6: **until** $\left| J(\theta^{(t)}) - J(\theta^{(t-1)}) \right| < \epsilon$
7: **return** $\theta^{(t)}$

Ring	Meaning
Outer rings	High cost (far from optimal)
Inner rings	Lower cost (getting closer)
Center point	Minimum (the best θ^*)



the contour plot
(bird's eye view: looking straight down)

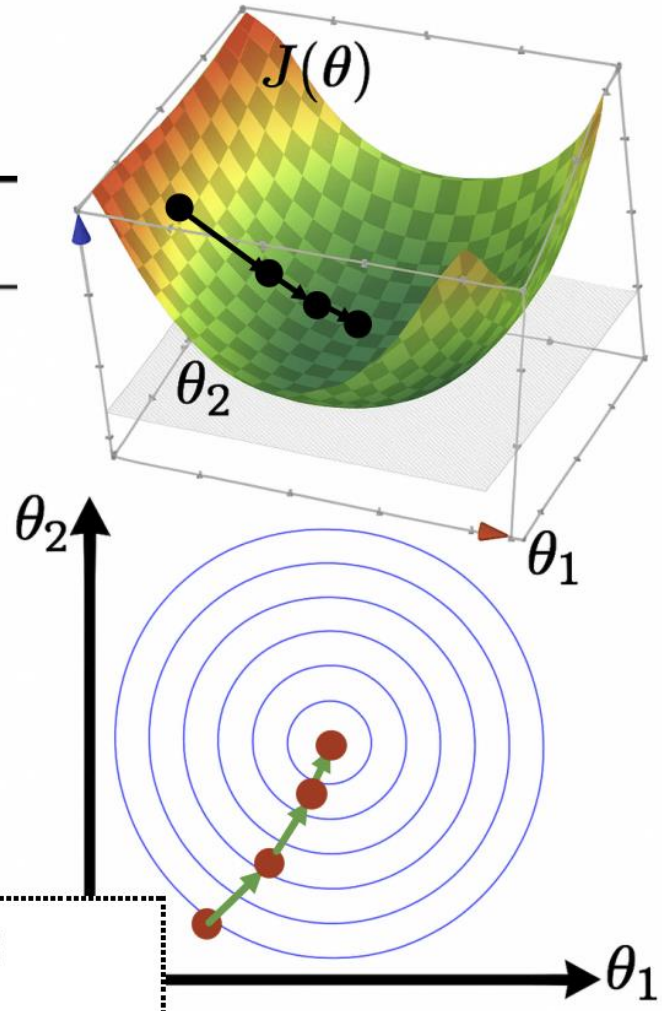
- Each ring: a contour line (same cost value)

Gradient Descent

Algorithm 1 Gradient Descent$(\theta_{\text{init}}, \eta, J, \epsilon)$

1: Initialize $\theta^{(0)} = \theta_{\text{init}}$
2: Initialize $t = 0$
3: **repeat**
4: $\quad t = t + 1$
5: $\quad \theta^{(t)} = \theta^{(t-1)} - \eta \nabla_\theta J(\theta^{(t-1)})$
6: **until** $\left| J(\theta^{(t)}) - J(\theta^{(t-1)}) \right| < \epsilon$
7: **return** $\theta^{(t)}$

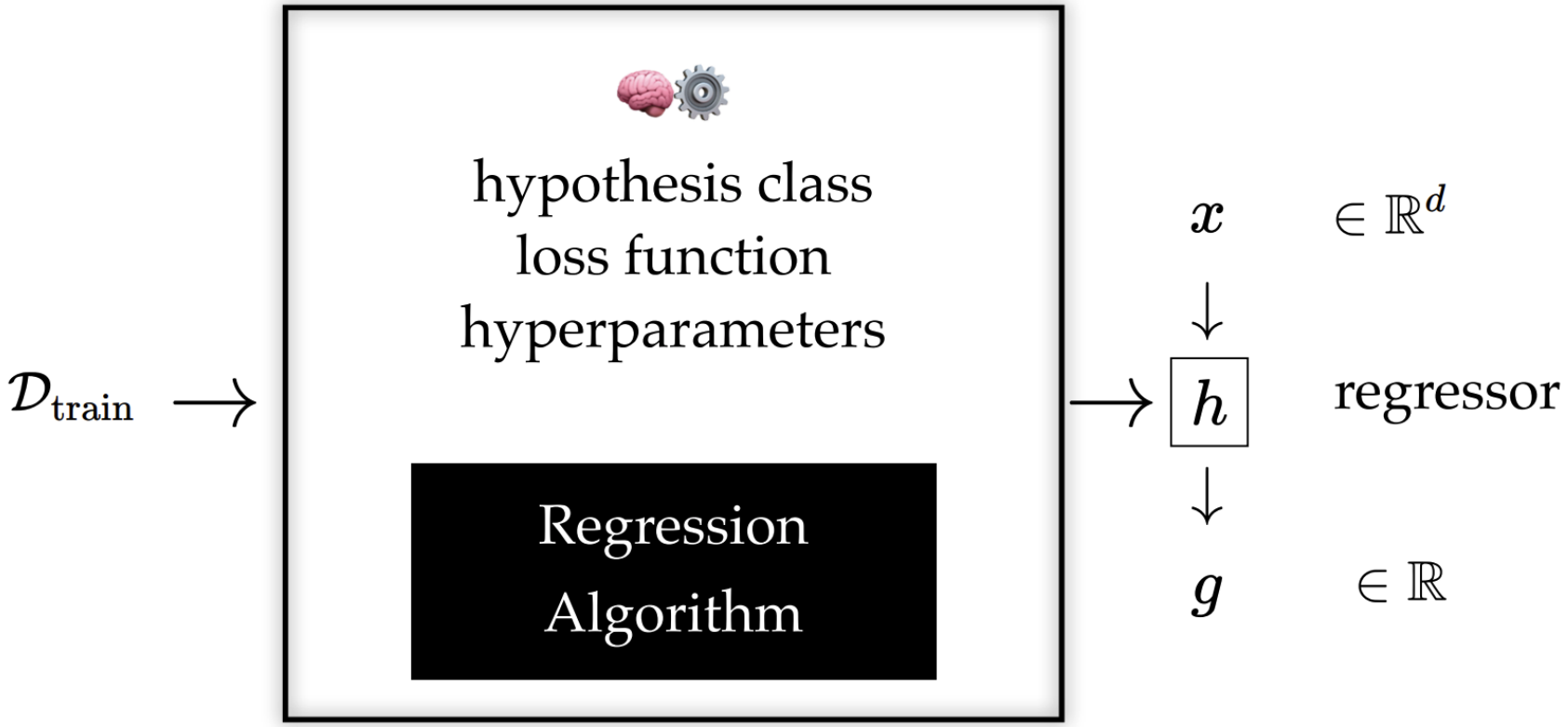
objective improvement is nearly zero.



Other possible stopping criterion for line 6:

- Small parameter change: $\|\theta^{(t)} - \theta^{(t-1)}\| < \epsilon$, or
- Small gradient norm: $\|\nabla_\theta J(\theta^{(t-1)})\| < \epsilon$

Regression



"Learn" a model $\longrightarrow$

train, optimize, tune, adapt ...

adjusting / updating / finding θ

gradient based

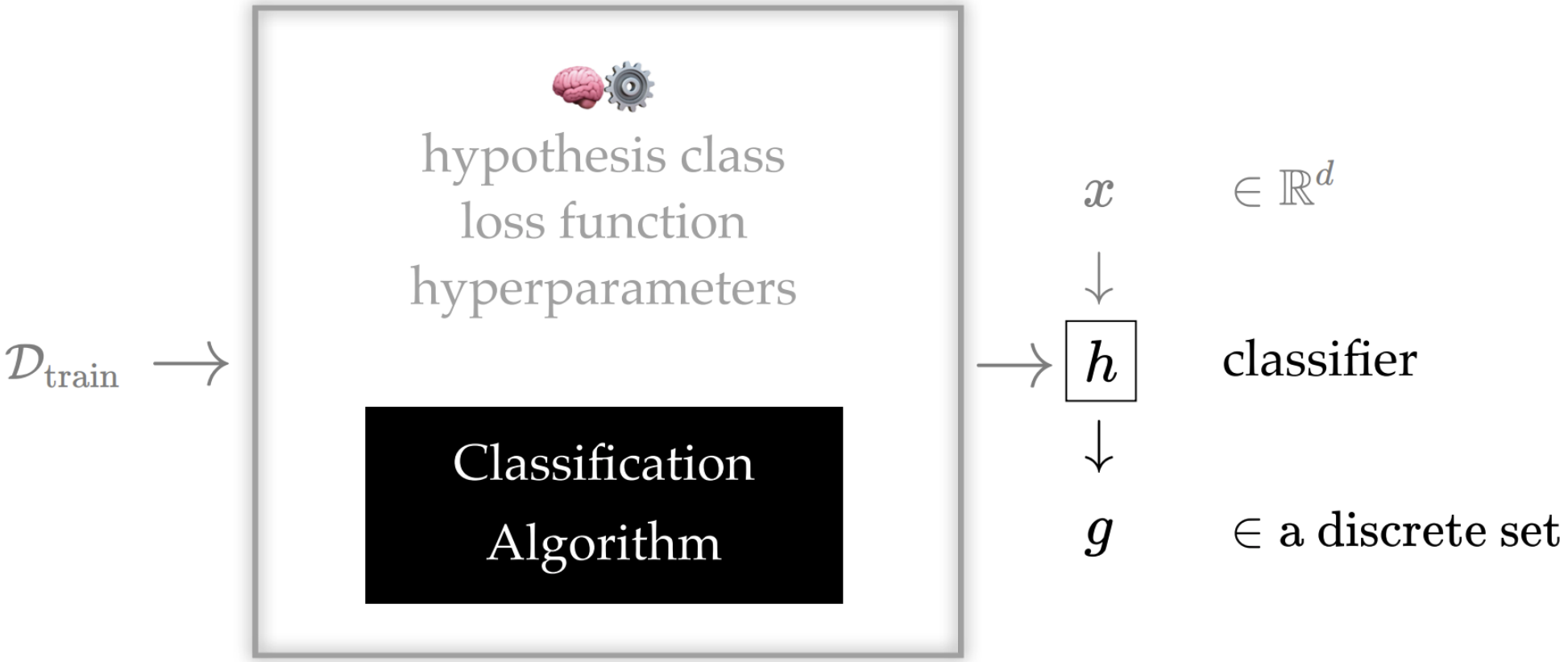
$\downarrow$ "Use" a model

predict, test, evaluate, infer ...

plug in the θ found

no gradients involved

Classification



- Classification is not a completely new problem.
  - Similar to the regression problem except the label space and the final prediction step.

$\{0, 1\}$

$\{😍, 🥺\}$

{"good", "better", "best", ...}

{"Fish", "Grizzly", "Chameleon", ...}

Classification

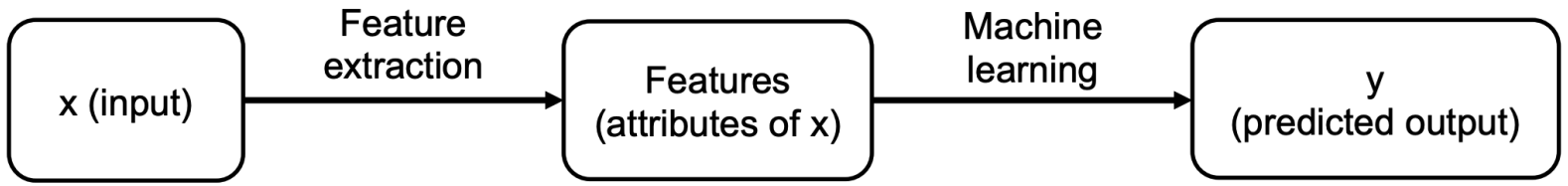
Examples of Machine Learning Classification in Real Life

Classification algorithms are widely used in many real-world applications across various domains, including:

- **Email spam filtering:** Filters our emails into spam or not spam based on email content.
- **Credit risk assessment:** Algorithms predict whether a loan applicant is likely to default by analyzing factors such as credit score, income and loan history. This helps banks make informed lending decisions and minimize financial risk.
- **Medical diagnosis**: Machine learning models classify whether a patient has a certain condition like cancer or diabetes based on medical data such as test results, symptoms and patient history. This aids doctors in making quicker, more accurate diagnoses, improving patient care.
- **Image classification :** Applied in fields such as facial recognition, autonomous driving and medical imaging.
- **Sentiment analysis:** Determining whether the sentiment of a piece of text is positive, negative or neutral. Businesses use this to understand customer opinions, helping to improve products and services.
- **Fraud detection :** Algorithms detect fraudulent activities by analyzing transaction patterns and identifying anomalies crucial in protecting against credit card fraud and other financial crimes.
- **Recommendation systems :** Used to recommend products or content based on past user behavior, such as suggesting movies on Netflix or products on Amazon. This personalization boosts user satisfaction and sales for businesses.

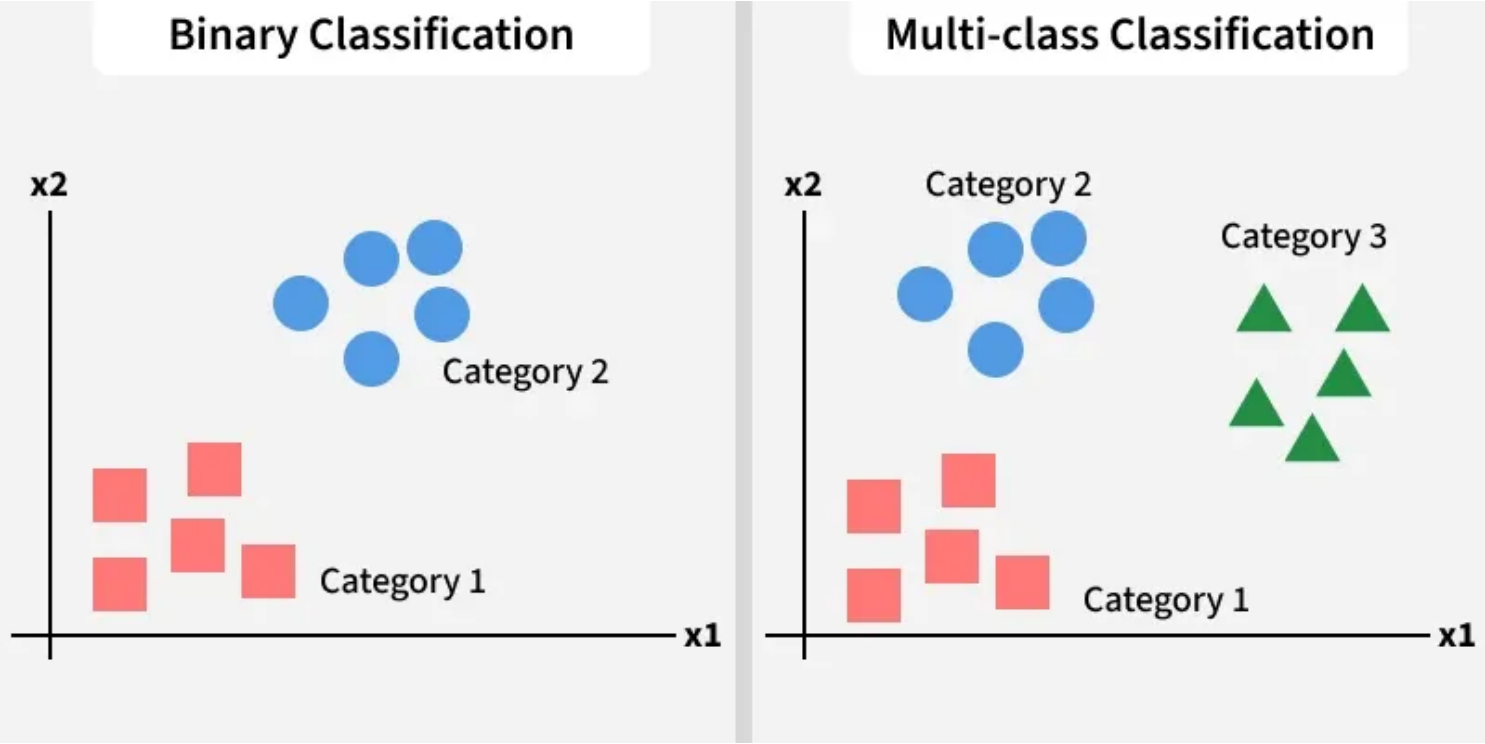
Classification

- Dataset:
  - Each data point, x, is associated with some label (aka class), y
- Goal of classification:
  - Given inputs x, write an algorithm to predict labels y
- Workflow of classification process:
  - Input is provided to you
  - Extract features from the input: attributes of the input that characterize each x and hopefully help with classification
  - Run some machine learning algorithm on the features
  - Output a predicted label y



Classification

- Binary classification: the answer can only be one of two values
  - Negative class
  - Positive class
- Multiclass classification



Outline

- Naïve binary classifiers
  - To use: separator
  - To learn: very difficult
- Logistic regression (binary class)
  - To use: sigmoid
  - To learn: negative log-likelihood loss
  - Extension: softmax regression (multiclass)

Naïve Binary Classification

- We have just learned linear regression
  - Compute $z = \theta_0 + \theta^T x \rightarrow$ that's our prediction
  - Works great for predicting house prices, temperatures, stock values -- anything continuous

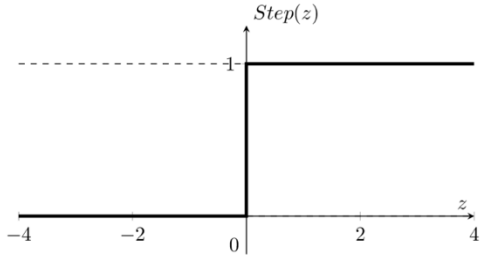
- How can we build a classifier (e.g., for spam classification)?
  - Can we directly use linear regression?
    - Compute $z = \theta_0 + \theta^T x$ on each email, and we get outputs like 2.7, -1.3, 0.4, 15.6...
    - But wait… Our labels are only 0 or 1
      - A prediction of 15.6 doesn't make any sense as a class label
      - We need a way to convert that continuous score into a binary decision

Naïve Binary Classification

- Can we start from linear regression?
- To use: separator – the simplest possible fix

	linear regressor	linear binary classifier
features	$x \in \mathbb{R}^d$	
label	$y \in \mathbb{R}$	$y \in \{0, 1\}$
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	
linear combination	$\theta^T x + \theta_0 = z$	
predict	$g = z$	$g = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$

- The problem is how to learn this θ



Step function

When z is positive → the model is "confident" that it's class 1

Naïve Binary Classification

- To learn: very difficult
  - We need a loss function to learn
  - One natural loss choice: $g = \text{step}(z) = \text{step}\left(\theta^\top x + \theta_0\right)$ y

$$\mathcal{L}_{01}(g, y) = \begin{cases} 0 & \text{if guess} = \text{label} \\ 1 & \text{otherwise} \end{cases}$$

  - Very intuitive, and easy to evaluate
    - You pay 0 for a correct prediction, 1 for a wrong one

very hard to optimize (NP-hard) 🥺

- "Flat" almost everywhere (zero gradient $\nabla_\theta J_{01}(\theta)$)
- "Jumps" elsewhere (no gradient) the function is discontinuous

Naïve Binary Classification

	linear regressor	linear binary classifier
features	$x \in \mathbb{R}^d$	
	$y \in \mathbb{R}$	$y \in \{0, 1\}$
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	
linear combo	$\theta^T x + \theta_0 \quad = z$	
predict	$g = z$	$g = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$ g is "flat" and discrete in θ
loss $\mathcal{L}(g, y)$	$\mathcal{L}_{\text{squared}} = (g - y)^2$	$\mathcal{L}_{01} = \begin{cases} 0 & \text{if } g = y \\ 1 & \text{otherwise} \end{cases}$ $\mathcal{L}_{01}(g, y)$ is "flat" and discrete in g
optimize method	• closed-form formula • gradient descent	training error almost "flat" w.r.t θ, gradient gives very little info

- Step function + 0-1 loss combination, while perfectly intuitive, cannot be optimized with gradient descent (NP-hard in general).

Can we find a smoother version of the step function that is differentiable and still maps to something between 0 and 1?

Linear regression gives you a score
→ you need a binary answer
→ the step function is the simplest converter
→ but it breaks gradient descent
→ so we smooth it into the sigmoid
→ and that's logistic regression!

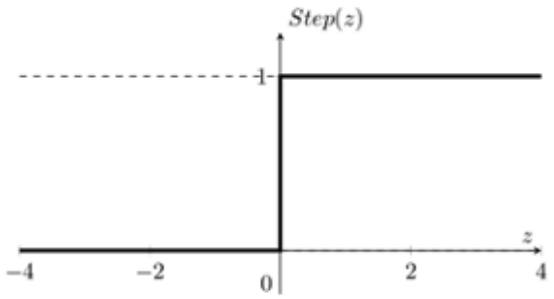
Logistic Regression

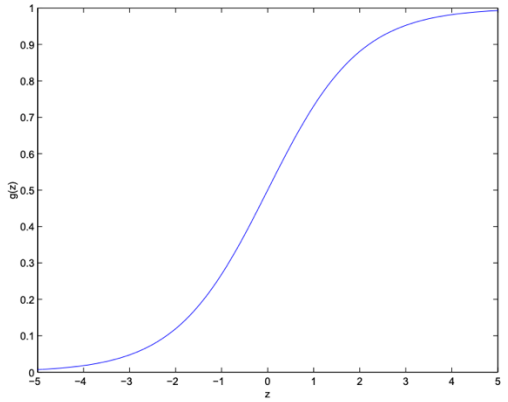
- Fix the prediction: the step function is flat and jumps

→ Replace it with a smooth version of the step function

🤔 There are many options of g

Idea: the sigmoid/logistic function $g(z) = \frac{1}{1+e^{-z}}$





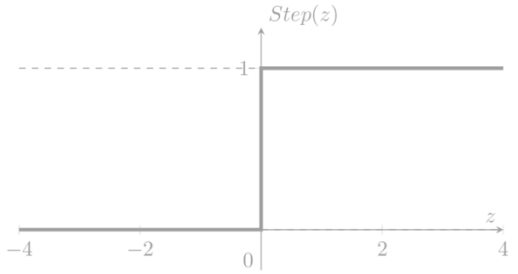
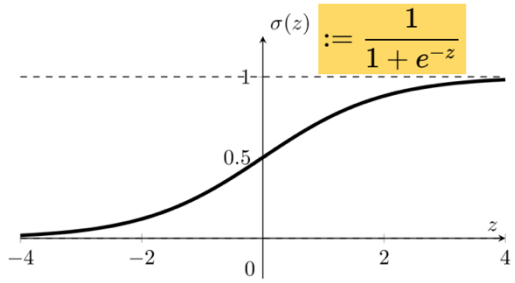
Smoothly increase from 0 to 1

- Example: if a spam-prediction model takes an email as input and outputs a value of 0.932, this implies a 93.2% probability that the email is spam

Logistic Regression

- Fix the prediction: the step function is flat and jumps

→ Replace it with a smooth version of the step function: sigmoid

	linear binary classifier	linear *logistic* binary classifier
features	$x \in \mathbb{R}^d$	
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	
linear combo	$\theta^T x + \theta_0 = z$	
predict	 $\begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$	 $\begin{cases} 1 & \text{if } \sigma(z) > 0.5 \\ 0 & \text{otherwise} \end{cases}$

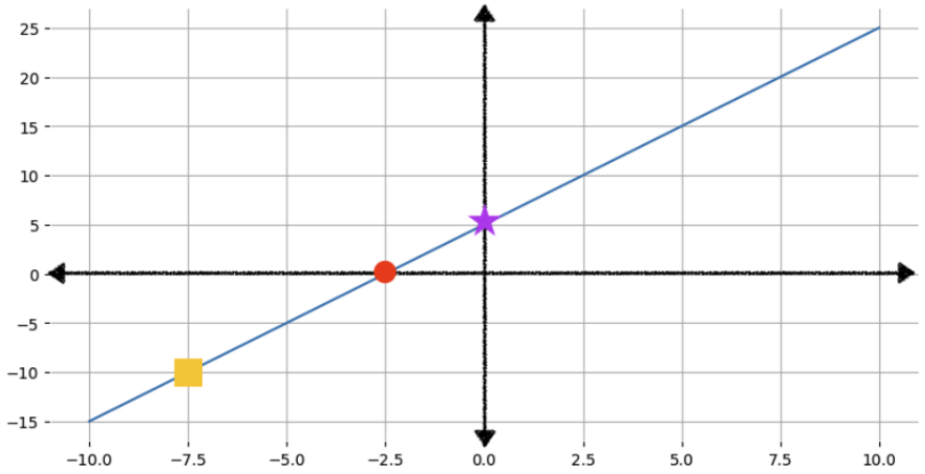
- A smooth approximation of the step function
  - Same shape
  - But differentiable everywhere & gradient information everywhere.

Sigmoid $\sigma(\cdot)$: *confidence* or *estimated likelihood* that x belongs to the positive class

Logistic Regression

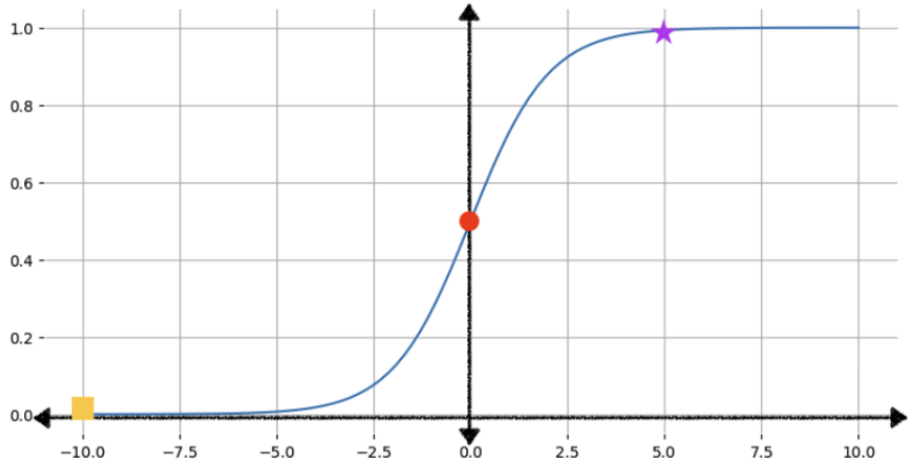
- Transform the linear output using the sigmoid function

$z = 2x + 5$



Graph of the linear function $z = 2x + 5$

$y' = 1 / (1 + e^{-z})$



Sigmoid curve
(after being transformed by the sigmoid function)

- A linear equation becomes input to the sigmoid function, which bends the straight line into an s-shape.
- The linear equation can output very big or very small values of z, but the output of the sigmoid function, y', is always between 0 and 1.
- Example: the yellow square on the left graph has a z value of –10, but the sigmoid function in the right graph maps that –10 into a y' value of 0.00004.

Logistic Regression

- Example
  - Task: predict if a user will purchase a subscription (y=1: buys; y=0: does not buy)
  - Features:
    - x_1: watched demo video or not (0/1)
    - x_2: number of active days in the last 7 days (0–7)
  - Model:
    - $z = b + w_1 x_1 + w_2 x_2$
    - $P(buy) = \sigma(z)$

Assume trained parameters: $b = -4, w_1 = 1.5, w_2 = 0.8$.
For a user with $x_1 = 1$ (watched video) and $x_2 = 4$ (active 4 days), what is his purchase probability?

Logistic Regression

- Example
  - Task: predict if a user will purchase a subscription (y=1: buys; y=0: does not buy)
  - Features:
    - x_1: watched demo video or not (0/1)
    - x_2: number of active days in the last 7 days (0–7)
  - Model:
    - $z = b + w_1 x_1 + w_2 x_2$
    - $P(buy) = \sigma(z)$

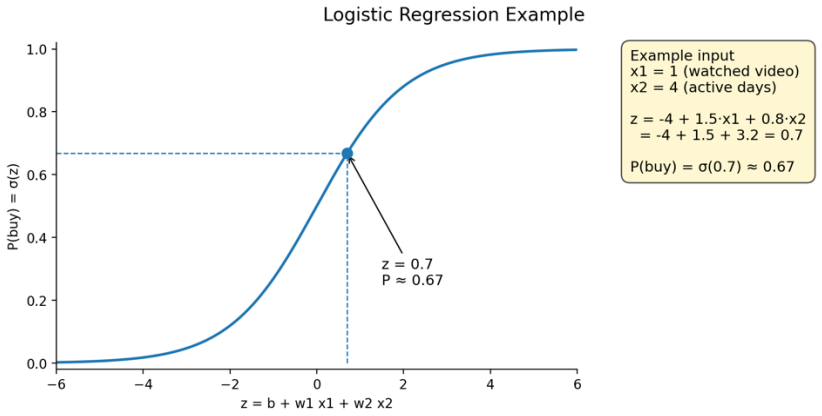
Assume trained parameters: $b = -4, w_1 = 1.5, w_2 = 0.8$.
For a user with $x_1 = 1$ (watched video) and $x_2 = 4$ (active 4 days), what is his purchase probability?

$$z = -4 + 1.5\times1 + 0.8\times4 = -4 + 1.5 + 3.2 = 0.7$$

$$\sigma(z = 0.7) \approx 0.67 > 0.5$$

Interpretation: this user has about 67% probability to buy

Thus, $P(buy) = 1$



Logistic Regression

- Fix the loss: the 0-1 loss is flat and discrete

→ Now that the prediction is a probability, we can use the probability language to define it

$$h_\theta(x) = \sigma(\theta^\top x + \theta_0) \in (0, 1)$$

Sigmoid

- Intuition: Suppose that the model says P(y=1)=0.9.
  - If the true label is y=1, the model did well
  - If the true label is y=0, the model did poorly
  - How do we quantify well and poorly? → through likelihood

want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y

Logistic Regression

- Fix the loss: want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y

$$h_\theta(x) = \sigma(\theta^\top x + \theta_0) \in (0, 1)$$ Outputs a number between 0 and 1

Sigmoid

Interpretation: $h_\theta(x) = P(y = 1|x; \theta)$

- Meaning: given parameters θ, when the model sees input x, what probability does it assign to the label being class 1?

As there are only 2 classes, $P(y = 1|x; \theta) + P(y = 0|x; \theta) = 1$

Therefore, $P(y = 0|x; \theta) = 1 - P(y = 1|x; \theta) = 1 - h_\theta(x)$

Logistic Regression

- Fix the loss: want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y

$$h_\theta(x) = \sigma(\theta^\top x + \theta_0) \in (0, 1)$$

Outputs a number between 0 and 1

Sigmoid

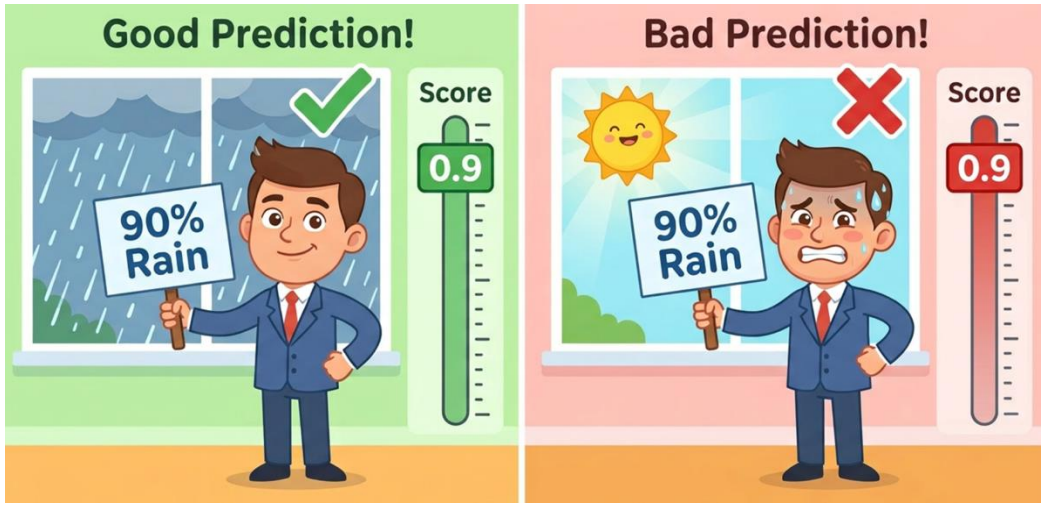
Example: suppose for some input x, the model computes $h_\theta(x) = 0.9$, then

	Probability
$P(y = 1 \mid x; \theta)$	$h_\theta(x) = 0.9$
$P(y = 0 \mid x; \theta)$	$1 - h_\theta(x) = 0.1$

If the true label is y = 1 → the model assigned probability 0.8 to the truth → good

If the true label is y = 0 → the model assigned probability 0.2 to the truth → bad

Reality is rain



Reality is sunny

Logistic Regression

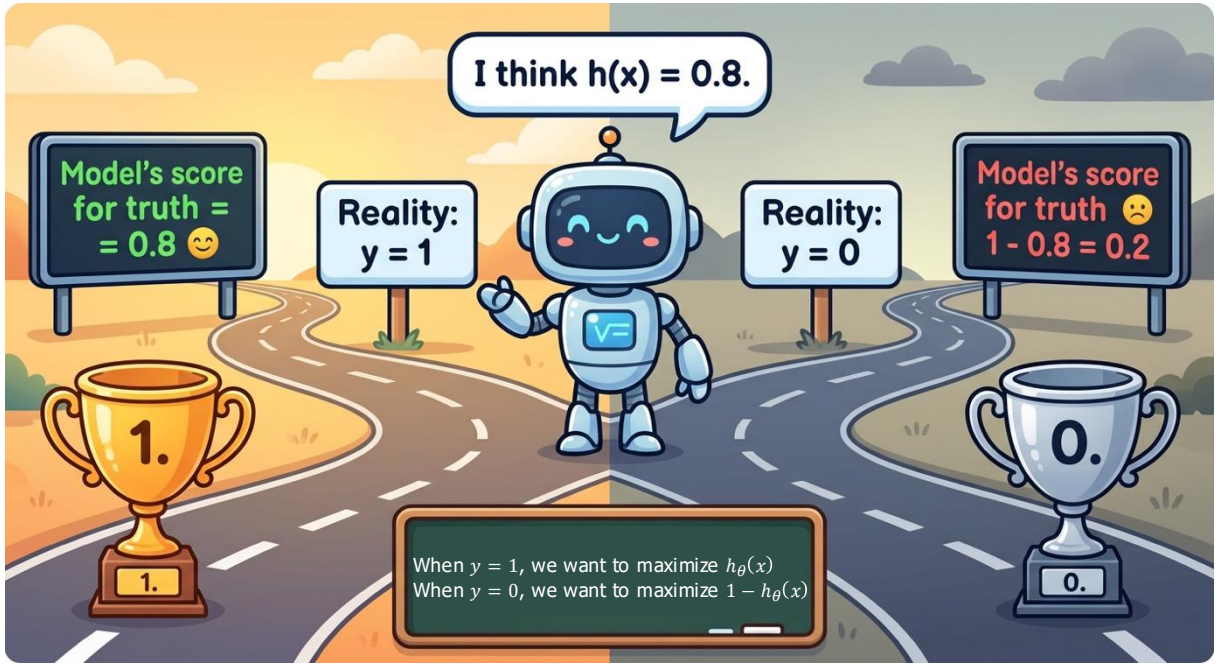
- Fix the loss: want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y

Given a single sample (x, y), how "likely" did our model think this outcome was?

$$P(y = 1 \mid x; \theta) = h_\theta(x)$$
$$P(y = 0 \mid x; \theta) = 1 - h_\theta(x)$$

- We want to maximize the probability the model assigns to the truth

True Label	What do we want to maximize?	Expressed in h_θ
$y = 1$	$P(y = 1 \mid x; \theta)$	$h_\theta(x)$
$y = 0$	$P(y = 0 \mid x; \theta)$	$1 - h_\theta(x)$



Logistic Regression

- Fix the loss: want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y

Given a single sample (x, y), how "likely" did our model think this outcome was?

$$P(y = 1 \mid x; \theta) = h_\theta(x)$$
$$P(y = 0 \mid x; \theta) = 1 - h_\theta(x)$$

- We want to maximize the probability the model assigns to the truth

True Label	What do we want to maximize?	Expressed in h_θ
$y = 1$	$P(y = 1 \mid x; \theta)$	$h_\theta(x)$
$y = 0$	$P(y = 0 \mid x; \theta)$	$1 - h_\theta(x)$

- Problem: tedious to write separate cases every time.
  Is there a single expression that automatically picks the right one?

→ Compact form: expresses the if-else logic

$$p(y \mid x; \theta) = (h_\theta(x))^y \, (1 - h_\theta(x))^{1-y}$$

Logistic Regression

- Fix the loss: want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y

Given a single sample (x, y), how "likely" did our model think this outcome was?

$$P(y = 1 \mid x; \theta) = h_\theta(x)$$
$$P(y = 0 \mid x; \theta) = 1 - h_\theta(x)$$

- We want to maximize the probability the model assigns to the truth

True Label	What do we want to maximize?	Expressed in h_θ
$y = 1$	$P(y = 1 \mid x; \theta)$	$h_\theta(x)$
$y = 0$	$P(y = 0 \mid x; \theta)$	$1 - h_\theta(x)$

→ Compact form: expresses the if-else logic

$$p(y \mid x; \theta) = (h_\theta(x))^y \, (1 - h_\theta(x))^{1-y}$$

  - y as a switch
    - When $y = 1$: the exponent of h turns on (becomes 1), the exponent of $1 - h$ turns off (becomes 0), i.e., $h^1 \cdot (1 - h)^0 = h$
    - When $y = 0$: the exponent of h turns off, the exponent of $1 - h$ turns on, i.e., $h^0 \cdot (1 - h)^1 = 1 - h$

Logistic Regression

- Fix the loss: want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y

Given a single sample (x, y), how "likely" did our model think this outcome was?

$$P(y = 1 \mid x; \theta) = h_\theta(x)$$
$$P(y = 0 \mid x; \theta) = 1 - h_\theta(x)$$

- We want to maximize the probability the model assigns to the truth

True Label	What do we want to maximize?	Expressed in h_θ
$y = 1$	$P(y = 1 \mid x; \theta)$	$h_\theta(x)$
$y = 0$	$P(y = 0 \mid x; \theta)$	$1 - h_\theta(x)$

→ Compact form: expresses the if-else logic

$$p(y \mid x; \theta) = (h_\theta(x))^y \, (1 - h_\theta(x))^{1-y}$$

a math trick to avoid writing if-else

- y as a switch
- Automatically picks the right probability based on the true label.

Logistic Regression

- Fix the loss: want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y

For n independent samples, the total likelihood is a product:

$$L(\theta) = P(y \mid X; \theta) = \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}; \theta) = \prod_{i=1}^{n} h_\theta(x^{(i)})^{y^{(i)}} (1 - h_\theta(x^{(i)}))^{1-y^{(i)}}$$

How likely is it that this model (with parameters θ) would produce the labels we actually observed?

- Products are bad: multiplying many small numbers → underflow; harder to differentiate than sums
- Take the log: $\ell(\theta) = \log L(\theta) = \sum_{i=1}^{n} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))$
  - log is monotonically increasing, maximizing ℓ is equivalent to maximizing L
  - When y=1: contribution is $\log h_\theta(x)$ → the closer the prediction is to 1, the larger this term is → good
  - When y=0: contribution is $\log(1 - h_\theta(x))$ → the closer the prediction is to 0, the larger this term is → good



Logistic Regression

- Fix the loss: want a smooth loss $\mathcal{L}(g, y)$ to reward g closer to y
  - Convention in ML: we typically minimize a loss. But likelihood should be maximized

  → Simple fix: flip the sign (minimize negative log-likelihood = maximize log-likelihood)
  - Replace the loss with negative log-likelihood (NLL)

$$\text{NLL}(\theta) = -\sum_{i=1}^{m} \left[y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right]$$

- Optimization: gradient descent

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} \ell(\theta) &= \left(y \frac{1}{g(\theta^T x)} - (1-y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\
&= \left(y \frac{1}{g(\theta^T x)} - (1-y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\
&= \left(y(1 - g(\theta^T x)) - (1-y) g(\theta^T x) \right) x_j \\
&= (y - h_\theta(x))\, x_j
\end{aligned}$$

← FYI. Not required for the exam

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)}$$

Logistic Regression

- Example: spam classification

Sample	True label y	Model prediction $h_\theta(x)$	Probability assigned to truth	$-\log(\cdot)$
Email A	1 (spam)	0.95	?	?
Email B	0 (not spam)	0.1	?	?
Email C	1 (spam)	0.2	?	?

Logistic Regression

- Example: spam classification

Sample	True label y	Model prediction $h_\theta(x)$	Probability assigned to truth	$-\log(\cdot)$
Email A	1 (spam)	0.95	0.95	0.05 (small loss)
Email B	0 (not spam)	0.1	0.9	0.11 (small loss)
Email C	1 (spam)	0.2	0.2	1.61 (large loss)

  - Email C: the model was only 20% confident in the correct answer, so -log(0.2) = 1.61 → a heavy penalty

→ The loss punishes confident wrong predictions severely

Logistic Regression

	linear binary classifier	linear *logistic* binary classifier
features	$x \in \mathbb{R}^d$	
label	$y \in \{0, 1\}$	
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	
linear combo	$\theta^T x + \theta_0 = z$	
predict	$\begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$	$\begin{cases} 1 & \text{if } g = \sigma(z) > 0.5 \\ 0 & \text{otherwise} \end{cases}$
loss	$\begin{cases} 0 & \text{if } g = y \\ 1 & \text{otherwise} \end{cases}$	$\begin{cases} -\log(g) & \text{if } y = 1 \\ -\log(1-g) & \text{if } y = 0 \end{cases}$ $\Leftrightarrow -[y \log g + (1-y)\log(1-g)]$
optimize via	NP-hard to learn	gradient descent

Negative log-likelihood

Logistic Regression

- Summary
  - Classification predicts a label from a discrete set; a linear binary classifier separates the feature space with a hyperplane defined by θ, θ_0.
  - The 0-1 loss is natural for classification but NP-hard to optimize.
  - The sigmoid $\sigma(z)$ gives a smooth, probabilistic step function; paired with the NLL loss, we can train via (S)GD.

Summary: Linear Regression & Logistic Regression

- Regression: Linear Regression
  - The output and the target are both real numbers: $h_\theta(x^{(i)}) \in \mathbb{R}$, $y^{(i)} \in \mathbb{R}$
  - Least square cost function for the i-th sample $(x^{(i)}, y^{(i)})$:

$$J^{(i)}(\theta) = \frac{1}{2}(h_\theta(x^{(i)}) - y^{(i)})^2$$

  - Mean-square cost function for the dataset

Note: Multiplying the cost function with a scalar will not change the local minima or global minima of the cost function

Summary: Linear Regression & Logistic Regression

- Classification: Logistic Regression (Binary)
  - Parameterized model $\bar{h}_\theta : \mathbb{R}^d \to \mathbb{R}$
  - Output probability $h_\theta(x) = g(\bar{h}_\theta(x)) = 1/(1+\exp(-\bar{h}_\theta(x))$
    - $g(\cdot)$: logistic/sigmoid function
      - Secret weapon:
        - Compresses any value between 0 and 1 → converting it into a probability

$$P(y=1 \mid x;\theta) = h_\theta(x)$$
$$P(y=0 \mid x;\theta) = 1 - h_\theta(x)$$

  - Negative log likelihood loss function

$$J^{(i)}(\theta) = -\log p(y^{(i)} \mid x^{(i)};\theta)$$ ➡ • $g(\cdot)$: logistic/sigmoid function

From Binary Classification to Multi-Class Classification

Component	Binary
Labels	y ∈ {0, 1}
Output	A single probability P(y=1|x)
Activation	Sigmoid
Loss	Binary cross-entropy

- In binary classification, one probability is enough as $P(y = 0) = 1 - P(y = 1)$

  Example: To predict {*hotdog, not_hotdog*} ← a scalar logit z suffices

  - ❑ Parameters: $\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$
  - ❑ Features: $x \in \mathbb{R}^d$



$\theta^T x + \theta_0$ → $z \in \mathbb{R}$ (Raw score for *hotdog*) → Normalizing (squashing) → $\sigma(z) \in (0,1)$

Implicitly determines

- $\sigma(\cdot)$: predicted probability that x belongs to *hotdog*
- $1 - \sigma(\cdot)$: predicted probability that x belongs to *not_hotdog*

From Binary Classification to Multi-Class Classification

- Classification: Logistic Regression (Binary)

Component	Binary
Labels	$y \in \{0, 1\}$
Output	A single probability $P(y=1\|x)$
Activation	Sigmoid
Loss	Binary cross-entropy

- What if $y \in \{1, 2, \ldots, K\}$?
  - Example: classifying handwritten digits $\rightarrow K = 10$

Softmax Regression for Multi-Class Classification

- Binary classification: one probability is enough as $P(y = 0|x) = 1 - P(y = 1|x)$
  - From Bernoulli distribution

- Multi-Class classification: a single scalar no longer suffices → we need to directly model the probabilities of a sample belonging to **every category** (where all these probabilities sum to 1)
  - From Multinomial distribution: $P(y|x)$ is a distribution over k possible discrete outcomes
    - Involves k numbers $\phi_1, \ldots, \phi_k$ specifying the probability of each of the outcomes
    - These numbers satisfy that $\sum_{i=1}^{k} \phi_i = 1$

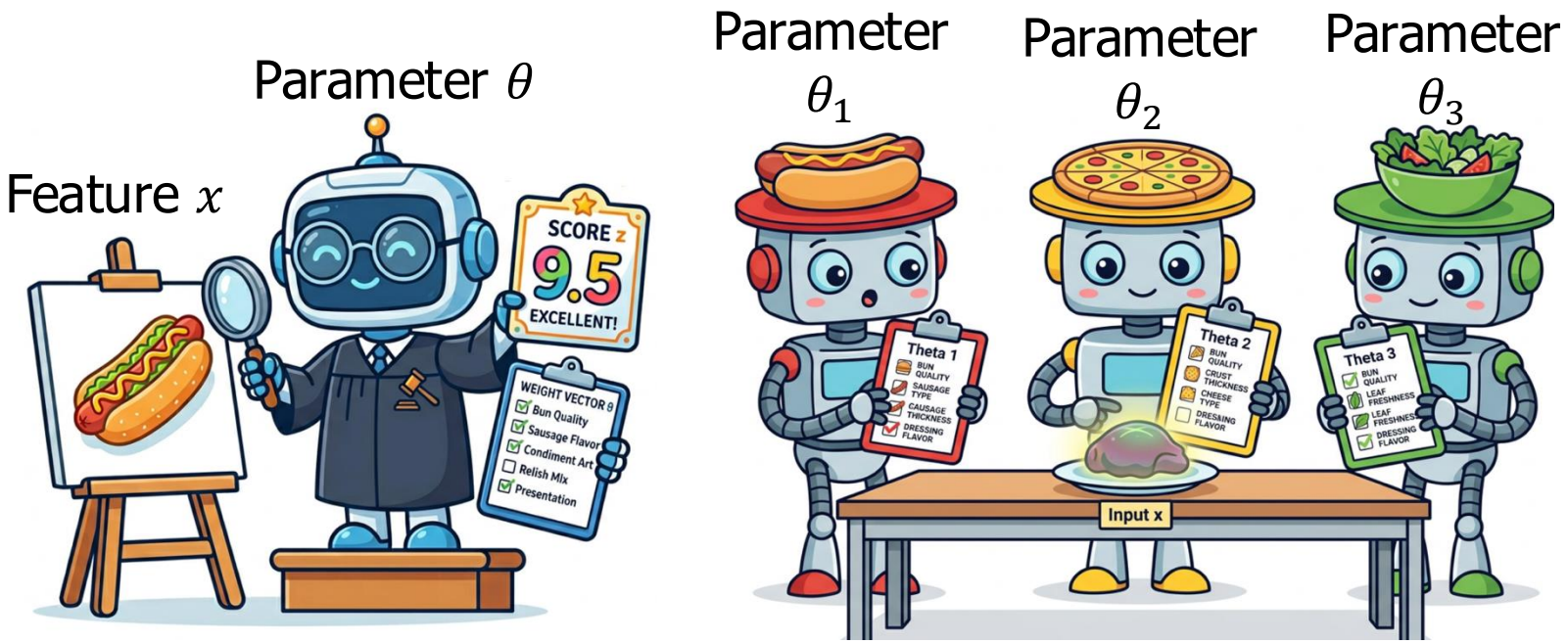
→ Design a parameterized model that outputs $\phi_1, \ldots, \phi_k$ satisfying this constraint given the input x

Softmax Regression

- Design a parameterized model that outputs $\phi_1, \ldots, \phi_k$ satisfying this constraint given the input x

	linear logistic *binary* classifier	one-out-of-K classifier
features	$x \in \mathbb{R}^d$	
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	$\theta \in \mathbb{R}^{d \times K}$, $\theta_0 \in \mathbb{R}^K$

- Introduce k groups of parameters $\theta_1, \ldots, \theta_k$ (each of them being a vector in $\mathbb{R}^d$: $\theta_i \in \mathbb{R}^d$)



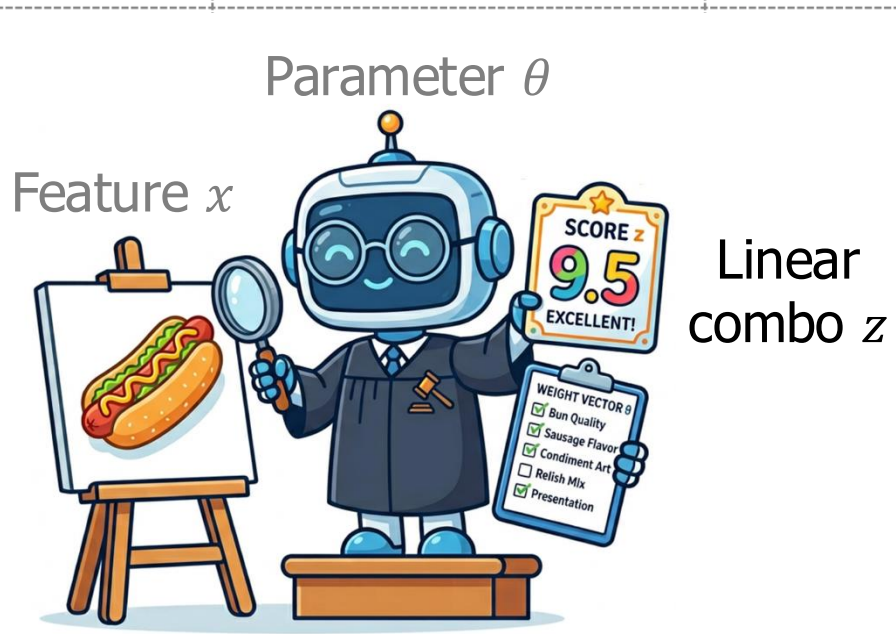
$$\theta = \begin{bmatrix} | & | & & | \\ \theta_1 & \theta_2 & \ldots & \theta_K \\ | & | & & | \end{bmatrix}$$

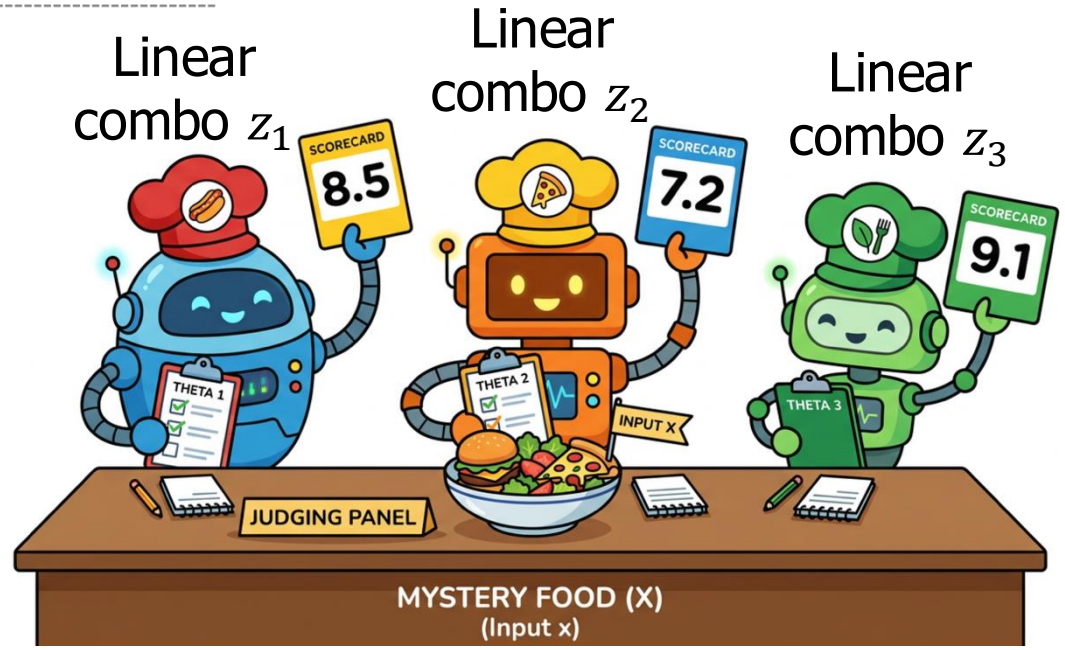
Softmax Regression

- Design a parameterized model that outputs $\phi_1, \ldots, \phi_k$ satisfying this constraint given the input x

	linear logistic *binary* classifier	one-out-of-K classifier
features	$x \in \mathbb{R}^d$	
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	$\theta \in \mathbb{R}^{d \times K}$, $\theta_0 \in \mathbb{R}^K$
linear combo	$\theta^T x + \theta_0 = z \in \mathbb{R}$	$\theta^T x + \theta_0 = z \in \mathbb{R}^K$

- Use $\theta_1^\top x, \ldots, \theta_k^\top x$ to represent $z_1, \ldots, z_k$





Softmax Regression

- Design a parameterized model that outputs ϕ_1, …, ϕ_k satisfying this constraint given the input x

	linear logistic *binary* classifier	one-out-of-K classifier
features	$x \in \mathbb{R}^d$	
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	$\theta \in \mathbb{R}^{d \times K}$, $\theta_0 \in \mathbb{R}^K$
linear combo	$\theta^T x + \theta_0 = z \in \mathbb{R}$	$\theta^T x + \theta_0 = z \in \mathbb{R}^K$

- Use $\theta_1^\top x, \ldots, \theta_k^\top x$ to represent z_1, …, z_k

$$z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_K \end{bmatrix}$$

- When we input a sample x, the model evaluates x using these K sets of weights separately, resulting in K different raw scores (logits).
- This K-dimensional vector z is a score leaderboard for these K categories.

Softmax Regression

- Design a parameterized model that outputs ϕ_1, ..., ϕ_k satisfying this constraint given the input x

	linear logistic *binary* classifier	one-out-of-K classifier
features	$x \in \mathbb{R}^d$	
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	$\theta \in \mathbb{R}^{d \times K}$, $\theta_0 \in \mathbb{R}^K$
linear combo	$\theta^T x + \theta_0 = z \in \mathbb{R}$	$\theta^T x + \theta_0 = z \in \mathbb{R}^K$

- Use $\theta_1^\top x, \ldots, \theta_k^\top x$ to represent z_1, ..., z_k

- Problem: we want to output the probabilities $P(y = 1 \mid x; \theta), \ldots, P(y = k \mid x; \theta)$
  - $\theta_i^T x$ is not necessarily within [0, 1]
  - The summation of $\theta_i^T x$'s is not necessarily 1

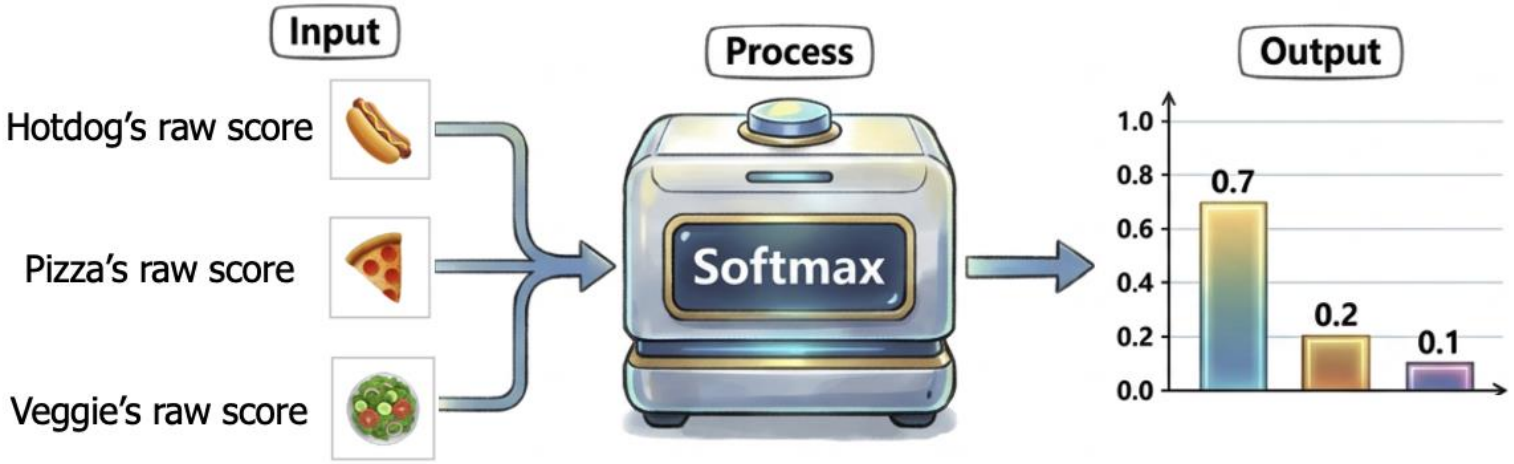
→ The Softmax function

$$\text{softmax}(z) := \begin{bmatrix} \frac{\exp(z_1)}{\sum_{k=1}^K \exp(z_k)} \\ \vdots \\ \frac{\exp(z_K)}{\sum_{k=1}^K \exp(z_k)} \end{bmatrix}$$

outputs all $\in [0, 1]$, sum to 1

Softmax Regression

- The Softmax function: $\mathbb{R}^k \rightarrow \mathbb{R}^k$



1. Calculate scores
2. Exponentiate
3. Normalize

Core Formula: $\mathrm{P}(y = i \mid x; \theta) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} = \frac{e^{\theta_i^T x}}{\sum_{j=1}^{K} e^{\theta_j^T x}}$

Softmax Regression

- Design a parameterized model that outputs $\phi_1, \ldots, \phi_k$ satisfying this constraint given the input x

	linear logistic *binary* classifier	one-out-of-K classifier
features	$x \in \mathbb{R}^d$	
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	$\theta \in \mathbb{R}^{d \times K}$, $\theta_0 \in \mathbb{R}^K$
linear combo	$\theta^T x + \theta_0 = z \in \mathbb{R}$	$\theta^T x + \theta_0 = z \in \mathbb{R}^K$
predict	$\sigma(z) = \frac{\exp(z)}{\exp(0) + \exp(z)}$	$\text{softmax}(z) = \begin{bmatrix} \frac{\exp(z_1)}{\sum_{k=1}^K \exp(z_k)} \\ \vdots \\ \frac{\exp(z_K)}{\sum_{k=1}^K \exp(z_k)} \end{bmatrix}$

- Use the output as the probabilities:

$$P(y = 1 \mid x; \theta), \ldots, P(y = k \mid x; \theta)$$

Softmax Regression

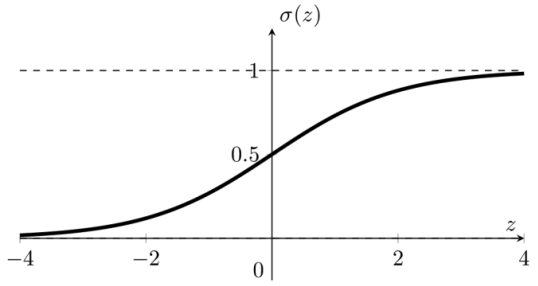
- Sigmoid and softmax look like two different formulas. Are they actually related?

sigmoid $\quad \mathbb{R} \to \mathbb{R}$

$$\sigma(z) := \frac{1}{1+\exp(-z)} = \frac{\exp(z)}{\exp(z)+\exp(0)}$$

implicit logit for the negative class

predict positive if $\sigma(z) > 0.5 = \sigma(0)$



softmax: $\quad \mathbb{R}^K \to \mathbb{R}^K$

$$\text{softmax}(z) := \begin{bmatrix} \frac{\exp(z_1)}{\sum_{k=1}^K \exp(z_k)} \\ \vdots \\ \frac{\exp(z_K)}{\sum_{k=1}^K \exp(z_k)} \end{bmatrix}$$

predict the category with the highest softmax score

⇔ argmax over K softmax scores

unifying rule: predict the class with the largest logit

Softmax Regression

	linear logistic *binary* classifier	one-out-of-K classifier
features	$x \in \mathbb{R}^d$	
parameters	$\theta \in \mathbb{R}^d, \theta_0 \in \mathbb{R}$	$\theta \in \mathbb{R}^{d \times K}$, $\theta_0 \in \mathbb{R}^K$
linear combo	$\theta^T x + \theta_0 = z \in \mathbb{R}$	$\theta^T x + \theta_0 = z \in \mathbb{R}^K$
predict	$\sigma(z) = \dfrac{\exp(z)}{\exp(0) + \exp(z)}$	$\text{softmax}(z) = \begin{bmatrix} \frac{\exp(z_1)}{\sum_{k=1}^K \exp(z_k)} \\ \vdots \\ \frac{\exp(z_K)}{\sum_{k=1}^K \exp(z_k)} \end{bmatrix}$
	predict positive if $\sigma(z) > \sigma(0)$	predict the class with the highest softmax score

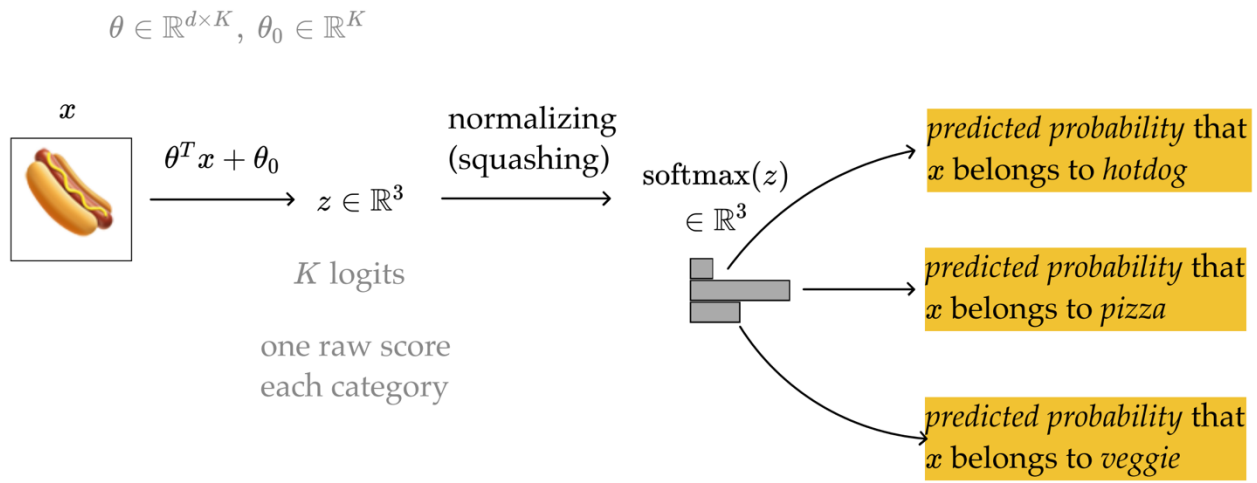
Softmax Regression

Component	Binary
Labels	y ∈ {0, 1}
Output	A single probability P(y=1|x)
Activation	Sigmoid
Loss	Binary cross-entropy

- What if y ∈ {1, 2, …, K}?
  - Example: classifying handwritten digits → K = 10

- In multi-class: for K classes, use K logit scores

  e.g., K=3: {hot-dog, pizza, veggie}



$$\text{softmax}(z) := \begin{bmatrix} \frac{\exp(z_1)}{\sum_{k=1}^K \exp(z_k)} \\ \vdots \\ \frac{\exp(z_K)}{\sum_{k=1}^K \exp(z_k)} \end{bmatrix}$$

outputs all $\in [0, 1]$, sum to 1

e.g.

$$\text{softmax}\left(\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}\right) = \begin{bmatrix} \frac{e^1}{e^1+e^2+e^3} \\ \frac{e^2}{e^1+e^2+e^3} \\ \frac{e^3}{e^1+e^2+e^3} \end{bmatrix} = \begin{bmatrix} 0.0900 \\ 0.2447 \\ 0.6653 \end{bmatrix}$$

max among the K logits

"soft" max'd in the output

Softmax Regression

Component	Binary	Multi-Class
Labels	y ∈ {0, 1}	y ∈ {1, 2, …, K}
Output	A single probability P(y=1|x)	K probabilities $P(y = i\|x)$
Activation	Sigmoid	Softmax
Loss	Binary cross-entropy	?

- In binary class: one probability is enough as $P(y = 0|x) = 1 - P(y = 1|x)$
  - ✓ To use: sigmoid
  - ✓ To learn: negative log-likelihood loss
- In multi-class: we need K probabilities: $P(y = 1|x), \dots, P(y = K|x)$, where $\sum_{k=1}^{K} P(y = k|x) = 1$
  - ✓ To use: softmax
    - We need a function that takes a bunch of raw scores and turns them into a valid probability distribution
  - To learn: ?
    - Representation: one-hot encoding
    - Loss: replace the loss from binary NLL to cross-entropy loss

Softmax Regression

- In binary class classification, we use the negative log-likelihood loss

$$\mathcal{L}_{\text{nll}}(g, y) = -\left[y \log g + (1 - y) \log (1 - g)\right]$$ "Switch"

  - $g = \sigma(z)$: the predicted probability of the positive class
  - $y \in \{0,1\}$: true label

  - If the true label is $y = 1$: loss = $-\log g$ (we want g to be large → loss is small)
  - If the true label is $y = 0$: loss = $-\log(1 - g)$ (we want g to be small → loss is small)

  ➢ It only takes the -log(predicted probability) of whichever class is the CORRECT one

  ➢ Goal: is to find a set of parameters θ that maximizes the likelihood of observing our actual training data
  = maximize the likelihood of observing our actual training data, given our model's current parameters θ
  = maximize the log-likelihood
  = minimize the negative log-likelihood

  → Just find the predicted probability of the CORRECT class, and take its negative log

- In multi-class classification, we *generalize* the negative log-likelihood loss
  - $g \in \mathbb{R}^K$: the softmax output (K predicted probabilities)

  → Negative log-likelihood of a single example (x, y): -log(predicted probability of the CORRECT class)

Softmax Regression

- How do we write one unified mathematical formula that can automatically pick out the probability of the correct class, no matter what the input is?

→ Negative log-likelihood K-class loss (aka, cross-entropy)

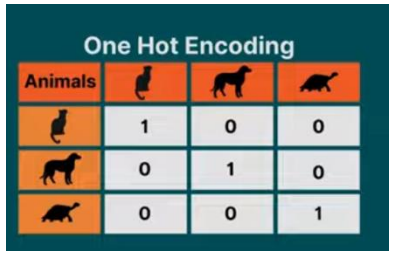
- g_k: probability/confidence of belonging in class k
- y_k: k-th entry in y (either 0 or 1)

$$\mathcal{L}_{\text{nllm}}(g, y) = -\sum_{k=1}^{K} y_k \cdot \log(g_k)$$

- g: softmax output
- y: one-hot encoding label

Softmax Regression

- One-hot encoding: generalizes from {0, 1} binary labels
  - Instead of using a word or a single integer y, we represent the true label as a vector:
    - Encode the K classes as an $\mathbb{R}^K$ vector
    - Put a single 1 (which means 'hot') at the index of the correct class, and 0s everywhere else



Training data

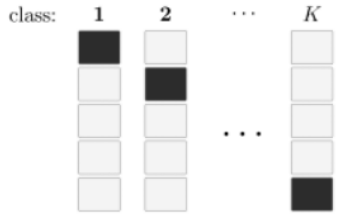
x	y
	"hot-dog"
	"pizza"
	"veggie"
	"veggie"

$K = 3$ $\longrightarrow$

Training data

x	y
	$\begin{bmatrix}1\\0\\0\end{bmatrix}$
	$\begin{bmatrix}0\\1\\0\end{bmatrix}$
	$\begin{bmatrix}0\\0\\1\end{bmatrix}$
	$\begin{bmatrix}0\\0\\1\end{bmatrix}$

in general, for K classes:



✓ Transformed our categories into a mathematical format → our new, generalized switch

Softmax Regression

- One-hot encoding: generalizes from {0, 1} binary labels (from scalar to vector)
  - Compatibility with algorithms: Many machine learning algorithms require *numerical input*.
    - ✓ It ensures that categorical variables are converted into a suitable format.

  - Eliminating ordinality: Many categorical variables have no inherent order (e.g., "Male" and "Female"). If we were to assign numerical values (e.g., Male = 0, Female = 1) the model might mistakenly interpret this as a ranking and lead to biased predictions.
    - ✓ One Hot Encoding eliminates this risk by treating each category independently and also enables mathematical elegance (cross-entropy loss).

sweaters

jeans





- 5 > 3:
→ jeans > sweaters

Label encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

One-Hot encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

Softmax Regression

- Negative log-likelihood K-class loss (aka, cross-entropy)

  - g_k: probability/confidence of belonging in class k
  - y_k: k-th entry in y (either 0 or 1)

$$\mathcal{L}_{\text{nllm}}(g, y) = -\sum_{k=1}^{K} y_k \cdot \log(g_k)$$

  - g: softmax output
  - y: one-hot encoding label

- Generalizes the negative log-likelihood loss
  - For any incorrect class k, the label $y_k = 0$: the entire terms becomes 0 → all the wrong classes vanish from the sum
  - For the single correct class c, the label $y_c = 1$: $-1 \cdot \log(y_c)$ is the only term that survives

→ Despite the K-term sum, only the term corresponding to its CORRECT class label contributes, since all other $y_k = 0$

  - Simply calculates the negative log of the model's predicted probability on the CORRECT class
  - The summation and the one-hot vector work together to automatically find that CORRECT class for us.

Softmax Regression

- Example:

current prediction
$g = \text{softmax}(\cdot)$

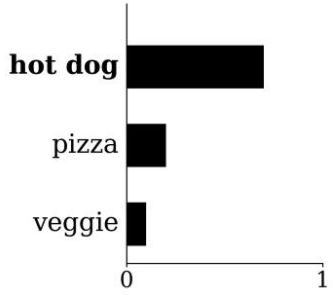
$\xrightarrow{-\log}$

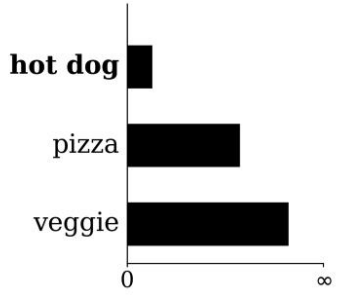
$-\log(g)$

y (true label)

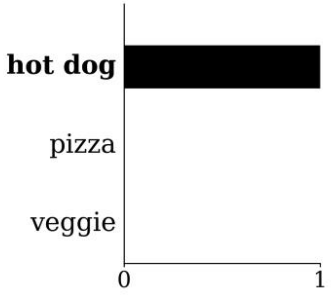
$$\mathcal{L}_{\text{nllm}} = -\sum_k y_k \cdot \log(g_k)$$

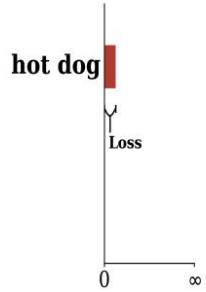






$\odot$





$= [0.7,\ 0.2,\ 0.1]$

$\approx [0.36,\ 1.61,\ 2.30]$

$= [1,\ 0,\ 0]$

$\text{Loss} \approx 0.36$

The model assigns 0.7 probability to hot-dog: not bad, but not perfect.

Element-wise multiplication

$$[1 \times 0.36,\ 0 \times 1.61,\ 0 \times 2.30] = [0.36,\ 0,\ 0]$$

The one-hot vector acts as a selector/mask: it zeros out all terms that don't correspond to the true class.

Summary: Classification

- ~~Naïve binary classifiers~~
  - ~~To use: separator~~
  - ~~To learn: very difficult~~
- Logistic regression (binary class)
  - To use: sigmoid
  - To learn: negative log-likelihood loss
- Softmax regression (multiclass)
  - To use: softmax
  - To learn: one-hot encoding, cross-entropy loss

Thanks